perplexity

building semester design system tell me everything it should provide in claude code right now

A strong design system is not just a component library—it is a **product decision system** that lets Claude Code generate consistent, accessible, responsive UI without inventing one-off colors, spacing, components, or interaction patterns.

For Claude Code, put the always-on rules in a root `CLAUDE.md`, keep canonical tokens/components in code, and add focused reusable skills for UI work. Claude Code reads project `CLAUDE.md` instructions at the start of sessions; Anthropic also supports reusable `SKILL.md` instruction packages for more specialized workflows. [1] [2]

What Semester should provide

Think of the system in six layers.

Layer	Semester should define	What Claude Code can do with it
Brand	Purpose, audience, voice, visual personality, anti-patterns	Make on-brand creative choices rather than generic SaaS UI
Foundations	Color, type, spacing, radii, elevation, motion, icons, breakpoints	Use stable primitives rather than arbitrary values
Semantic tokens	Roles such as `surface`, `text-primary`, `border-subtle`, `action-primary`	Support themes and visual updates without mass refactors
Components	API, states, variants, composition rules, accessibility behavior	Build screens from known parts instead of creating duplicates
Patterns	Forms, data views, navigation, onboarding, settings, empty/loading/error states	Solve repeated UX problems consistently
Governance	Quality checks, contribution workflow, deprecation rules, release/versioning	Keep the system coherent as the product grows

The key distinction: **primitive tokens** express raw values; **semantic tokens** express meaning. Components should almost always consume semantic tokens.

```
/* Avoid */
.card {
  background: #ffffff;
  color: #111827;
  border: 1px solid #e5e7eb;
  border-radius: 12px;
}
```

```
/* Prefer */
.card {
  background: var(--color-surface-raised);
  color: var(--color-text-primary);
  border: 1px solid var(--color-border-subtle);
  border-radius: var(--radius-lg);
}
```

That lets “Semester” evolve from one theme or brand expression to another by changing the token mapping—not every individual component.

Foundations to define

Brand and product direction

Give Claude an explicit design brief, including:

- What Semester is and who it serves.
- Primary product surfaces: marketing site, authenticated application, dashboard, editor, marketplace, documentation, mobile web, etc.
- Brand attributes—e.g. precise, editorial, warm, technical, playful, institutional, high-trust.
- The visual reference point: “quiet editorial software,” “high-density developer tool,” “premium consumer productivity app,” and so on.
- Explicit non-goals: no gradient-heavy glassmorphism, no random icon cards, no pill-everything interfaces, no excessive rounded containers, no arbitrary decoration.
- Content voice: sentence case versus title case, direct vs. playful microcopy, preferred terminology, capitalization rules, and banned language.
- Dark mode strategy: supported now, later, or intentionally unsupported.
- Internationalization assumptions: locales, text expansion, RTL needs, date/time and numeric formatting.
- Accessibility target: at minimum WCAG 2.2 AA, with clear expectations for keyboard support, focus visibility, contrast, reduced motion, errors, and screen-reader labels.

This context is important because a system can be technically consistent while still producing the wrong kind of product. Claude’s persistent project instructions are specifically intended to include architecture, conventions, commands, and common workflows. [1]

Token system

Define tokens in three levels:

1. **Reference tokens**: raw palette and scale values.
2. **Semantic tokens**: meaningful UI roles.
3. **Component tokens**: optional component-specific roles where genuinely needed.

Your initial token families should include:

- **Color**
  - Brand/accent scale
  - Neutral scale
  - Success, warning, danger, information
  - Surfaces: canvas, base, raised, overlay, inverse
  - Text: primary, secondary, muted, disabled, inverse, link
  - Borders: subtle, default, strong, focus, destructive
  - Interactive: default, hover, pressed, selected, disabled
- **Typography**
  - Font families and fallbacks
  - Type scale
  - Font weights
  - Line-height and letter-spacing
  - Fluid/responsive display styles if needed
  - Named semantic styles such as `body-sm`, `body-md`, `label-sm`, `heading-2xl`, `code-sm`
- **Spacing**
  - A constrained scale such as `0, 1, 2, 3, 4, 6, 8, 10, 12, 16, 20, 24`
  - Rules for when exceptions are allowed
- **Sizing**
  - Control heights
  - Icon sizes
  - Touch-target minima
  - Content-width and layout container sizes
- **Shape**
  - Border-widths
  - Radius scale
  - Avatar sizes
- **Elevation**
  - Surface levels
  - Shadow recipes
  - Overlay/modal treatment
- **Motion**
  - Durations, easing, interaction motion rules
  - Reduced-motion equivalents
- **Responsive layout**

    - Breakpoints
    - Grid columns
    - Page gutters
    - Max-widths
    - Stack-to-grid rules
- **Z-index**
    - A named, fixed layering ladder: base, sticky, dropdown, popover, modal, toast, tooltip
- **Opacity and effects**
    - Disabled/overlay opacity
    - Blur rules if used
    - Focus-ring treatment

Avoid raw CSS literals outside the token source unless the system has explicitly permitted them. That should be a non-negotiable Claude Code rule.

Layout grammar

A design system should provide the rules that make pages feel related:

- Global page gutters for compact, tablet, desktop, and wide desktop.
- Standard reading/content widths.
- Application-shell structure: sidebar, top bar, content region, panel region.
- Grid primitives: stack, inline, cluster, grid, split pane, center, container.
- Density modes, if Semester needs them: comfortable, standard, compact.
- Standard section rhythm: page title → context/actions → primary surface → supporting content.
- Alignment rules for labels, controls, icons, tables, and actions.
- Responsive priorities: which content stacks, hides, scrolls, truncates, or becomes a drawer.
- Safe-area handling for mobile and fixed navigation.
- Empty-state placement, loading skeleton layout, and error-state layout.

A small layout-primitives package prevents Claude from repeatedly hand-authoring subtly different flex/grid code.

Components and patterns

Essential components

Each component needs more than visual examples. Define:

- Intended use and non-use cases.
- Import path and implementation status.
- Variants and controlled props.
- Allowed composition patterns.
- All states: default, hover, focus-visible, active, selected, disabled, loading, invalid, read-only.
- Responsive behavior.
- Accessibility semantics and keyboard behavior.
- Content constraints: label lengths, icon placement, truncation, localization behavior.
- Test and Storybook expectations.

Start with the smallest durable set:

- Button, IconButton, Link, ButtonGroup.
- Text, Heading, Code, Label, Caption.
- Icon, Avatar, Badge, Status, Tag/Chip.
- Input, Textarea, Select/Combobox, Checkbox, RadioGroup, Switch, Slider, Date/Time input where needed.
- Field wrapper: label, description, required marker, validation, error text.
- Alert, Toast, Banner, InlineMessage.
- Card, Surface, Divider, List, KeyValue.
- Tabs, Breadcrumbs, Pagination, Stepper.
- Menu, Dropdown, Popover, Tooltip, Context Menu.
- Dialog, AlertDialog, Drawer, Sheet.
- Table/DataGrid, Filter bar, Search input, Sort controls.
- Skeleton, Spinner, Progress, EmptyState, ErrorState.
- Navigation: top nav, sidebar nav, mobile nav, nav group/item.
- Page header, section header, toolbar, action bar.

Product patterns

Components alone do not solve product consistency. Semester should also ship opinionated patterns for:

- Authentication: sign in, registration, password reset, invite acceptance, MFA.
- Onboarding: welcome, progressive setup, checklists, empty initial workspace.

- Forms: create/edit, validation timing, unsaved changes, autosave, destructive confirmation.
- Search and discovery: command palette, global search, filtering, no-results state.
- Data-heavy screens: list/detail, tables, row actions, bulk selection, saved views.
- CRUD lifecycle: create, success confirmation, edit, archive/delete, undo.
- Feedback: optimistic updates, loading, errors, retry, offline/degraded state.
- Permissions: restricted view, missing access, role explanation, request-access flow.
- Settings: organization/workspace/profile billing patterns.
- Notifications: in-app, toast, inbox, preferences.
- Mobile adaptation: priority content, navigation, drawers, sticky actions, touch targets.

For each pattern, document the preferred component composition, behavior, edge cases, and a production example.

Claude Code package

Your immediate goal is to make the system **machine-usable**. A good repository shape:

```
semester/
├── CLAUDE.md
├── README.md
├── docs/
│   ├── foundations.md
│   ├── content-and-voice.md
│   ├── accessibility.md
│   ├── responsive.md
│   ├── component-status.md
│   ├── patterns/
│   │   ├── forms.md
│   │   ├── data-views.md
│   │   ├── feedback.md
│   │   └── navigation.md
│   └── decisions/
├── packages/
│   ├── tokens/
│   ├── icons/
│   ├── ui/
│   └── eslint-config/
├── apps/
│   └── web/
├── .claude/
│   └── skills/
│       ├── build-ui/
│       │   └── SKILL.md
│       ├── audit-ui/
│       │   └── SKILL.md
│       └── create-component/
│           └── SKILL.md
└── .storybook/
```

Use `CLAUDE.md` for short, always-applicable instructions. Claude Code supports root or `.claude/CLAUDE.md` project memory files and supports reusable skills that live in `SKILL.md` files. [2] [1]

Starter `CLAUDE.md`

```
# Semester Design System

## Mission
Semester is the sole source of truth for UI foundations, React components,
product interaction patterns, and accessibility rules.

Build interfaces that feel deliberate, calm, information-rich, and native to
Semester. Favor clear hierarchy and useful content over decorative UI.

## Non-negotiable rules
- Use only exports from `@semester/ui`, `@semester/tokens`, and `@semester/icons`
  before creating new UI primitives.
- Never use arbitrary hex/rgb/hsl colors, pixel spacing values, radii, shadows,
  z-index values, font sizes, or animation timings in product UI.
- Use semantic tokens, not raw palette tokens, in components and application code.
- Do not add a new component until existing components and compositions have been eva
- Do not modify generated token files manually.
- Do not use emoji as interface icons. Use `@semester/icons`.
- Do not introduce a dependency for UI without explicit approval.
- Preserve keyboard operation, visible focus, semantic HTML, and WCAG 2.2 AA contras
- Support narrow screens first; do not hide critical workflows on mobile.
- Respect `prefers-reduced-motion`.

## UI workflow
1. Inspect the nearest relevant screen and existing component stories.
2. Search `@semester/ui` for a suitable component or documented composition.
3. Write a short implementation plan for new flows or new primitives.
4. Implement with tokens and existing primitives.
5. Cover loading, empty, error, disabled, and permission-restricted states.
6. Run typecheck, lint, unit tests, and Storybook checks.
7. Summarize changed files, reused system elements, and accessibility decisions.

## Component API principles
- Prefer composable primitives over large boolean-prop APIs.
- Use `variant`, `size`, and `tone` only when they map to documented design choices.
- Defaults must be safe, accessible, and visually standard.
- Every interactive component needs a disabled and loading strategy where relevant.
- Component names are singular PascalCase; props use camelCase.
- Use `data-slot` for stylable component subparts where needed.

## Accessibility
- Use native HTML controls whenever possible.
- All icon-only controls require an accessible name.
- Error messages must be programmatically associated with their field.
- Dialogs must trap focus, restore focus on close, and close on Escape unless unsafe
- Do not communicate meaning through color alone.
- Minimum interactive target: 44 by 44 CSS pixels where practical.

## Verification
```

```
Run the commands in package.json appropriate to touched packages.
For visual changes, add or update Storybook stories for every new variant and state.
```

High-value Claude skills

Create three initial skills:

- `build-ui`: Builds a feature using existing Semester components, requires state coverage and responsive behavior, and prohibits new primitives without justification.
- `audit-ui`: Reviews a route or component for token violations, accessibility, hierarchy, responsiveness, and inconsistent patterns.
- `create-component`: Creates a component only after documenting purpose, API, states, accessibility contract, stories, tests, and adoption example.

A short `build-ui/SKILL.md` might say:

```
---
name: build-ui
description: Build or revise an interface using the Semester design system.
---

Before editing:
1. Read CLAUDE.md and relevant docs in docs/patterns.
2. Inspect existing components and their Storybook stories.
3. State which Semester components and patterns will be used.

Implementation requirements:
- Reuse established components first.
- Use semantic design tokens only.
- Include loading, empty, error, disabled, and narrow-screen behavior where applicabl
- Keep visual hierarchy simple: one primary action per scope.
- Do not create a new primitive unless the existing system cannot support the need.

Before completion:
- Check keyboard navigation and focus visibility.
- Check labels, descriptions, errors, and icon-only controls.
- Run typecheck, lint, and relevant tests.
- Report token/component reuse and any proposed system gap.
```

Build order

Do **not** begin by producing 80 components. Build in this order:

1. Write the brand, product, content, accessibility, and responsive rules.
2. Build and validate tokens—including semantic tokens and light/dark mappings if dark mode matters.
3. Add layout primitives and application shell.
4. Ship typography, buttons, links, icons, fields, feedback, overlays, and navigation.
5. Establish page, form, data-view, and empty/loading/error patterns.

6. Build only the domain-specific components Semester repeatedly needs.
7. Add Storybook examples, tests, lint rules, visual regression checks, and an adoption/migration guide.
8. Measure system health: percentage of UI using system components, number of token violations, duplicate component count, accessibility failures, and time required to ship a typical screen.

For every feature, define "done" as: visual system compliance, functional behavior, responsive behavior, keyboard support, semantic markup, all meaningful states, tests, and documentation when the system changes.

Governance and guardrails

Semester needs mechanisms that stop entropy:

- **Source of truth:** Specify whether tokens live in CSS variables, TypeScript, JSON, Style Dictionary, or another pipeline—and which artifact is generated.
- **Ownership:** Name a design-system owner or review group for foundational changes.
- **Contribution template:** Problem, users affected, evidence, proposed API, alternatives considered, accessibility notes, migration impact.
- **Deprecation policy:** Never silently break a component API; mark deprecated, document migration, provide a removal version/date.
- **Visual regression testing:** Capture stories and critical product routes.
- **Automated checks:** Reject raw colors, unauthorized spacing values, unapproved z-index values, inline SVGs when icon components exist, and inaccessible interactive markup.
- **Component maturity:** Track `experimental`, `beta`, `stable`, and `deprecated` states.
- **Changelog:** Record new tokens, component changes, behavior changes, and migrations.
- **Decision records:** Preserve important choices—such as why you use 4px vs. 8px base spacing, or whether destructive actions use red buttons—so Claude does not reopen settled decisions.

The goal is that Claude Code can answer, before it writes code: *Which existing primitive solves this? Which semantic token expresses its role? What states and accessibility constraints apply? How do I verify the result?* That is the threshold at which Semester becomes a real system rather than a styling folder.

⁂

Build a complete Claude Code design system config: CLAUDE.md templates, token schemas, Figma MCP setup, and SKILL.md rules

Computer
Claude Code design-sync audit: how to map your React component library and design tokens so Claude stops generating generic UI
Computer
Show me an example CLAUDE.md for this design system
How should I structure the semantic tokens for Semester
How do I write a SKILL.md for frontend UI generation
Sources
15

Below is a production-oriented starting configuration for **Semester**: a token-first React design system that Claude Code can reliably use, audit, and extend without falling back to generic UI.

It uses four complementary mechanisms:

- Root `CLAUDE.md` for durable rules that should apply to every coding task.
- Typed semantic token files as the implementation source of truth.
- Figma MCP for reading variables, components, layout context, and selected frames in Figma.
- Task-specific `SKILL.md` files for building UI, auditing drift, and adding components. Figma's recommended Claude Code setup is its official plugin, which bundles the MCP configuration and workflow skills; Figma MCP also exposes design context including components, variables, layout information, FigJam content, and Code Connect mappings. [16] [17]

Repository structure

Use a structure where tokens and components are code-owned, Figma is a design input, and written rules are compact enough for Claude to apply consistently.

```
semester/
├── CLAUDE.md
├── package.json
├── pnpm-workspace.yaml
├── .mcp.json                          # Optional project-scoped MCP definition
├── .claude/
│   ├── settings.json
│   └── skills/
│       ├── build-ui/
│       │   ├── SKILL.md
│       │   └── references/
│       │   │   ├── component-inventory.md
│       │   │   ├── screen-checklist.md
```

```
│       │   │   └── figma-workflow.md
│       ├── audit-design-sync/
│       │   ├── SKILL.md
│       │   └── references/
│       │       └── audit-report-template.md
│       └── create-component/
│           ├── SKILL.md
│           └── references/
│               └── component-rfc-template.md
├── docs/
│   ├── architecture.md
│   ├── design-principles.md
│   ├── accessibility.md
│   ├── content-style.md
│   ├── figma-mapping.md
│   ├── component-inventory.md
│   ├── patterns/
│   │   ├── application-shell.md
│   │   ├── forms.md
│   │   ├── feedback.md
│   │   ├── navigation.md
│   │   └── data-display.md
│   └── decisions/
│       ├── 001-token-architecture.md
│       └── 002-component-api.md
├── packages/
│   ├── tokens/
│   │   ├── src/
│   │   │   ├── primitives.ts
│   │   │   ├── semantic.ts
│   │   │   ├── components.ts
│   │   │   ├── themes/
│   │   │   │   ├── light.ts
│   │   │   │   └── dark.ts
│   │   │   ├── index.ts
│   │   │   └── validate.ts
│   │   ├── dist/
│   │   └── package.json
│   ├── icons/
│   ├── ui/
│   │   ├── src/
│   │   │   ├── components/
│   │   │   ├── primitives/
│   │   │   └── styles/
│   │   ├── stories/
│   │   └── package.json
│   └── eslint-config/
├── apps/
│   ├── web/
│   └── storybook/
└── scripts/
    ├── audit-token-usage.ts
    └── validate-design-system.ts
```

The important hierarchy is:

```
Raw values → reference tokens → semantic tokens → component tokens → components → pro
```

Application code should primarily consume **semantic tokens and components**. It should not choose colors, radii, shadows, or spacings ad hoc.

Root CLAUDE.md

Place this at the repository root as `CLAUDE.md`. Claude Code supports persistent project instructions in `CLAUDE.md`; its Skills system loads more focused `SKILL.md` files dynamically when the work is relevant, so reserve the root file for universal rules rather than stuffing every implementation detail into it.[18] [19]

```
# Semester Design System — Claude Code Instructions

## System purpose

Semester is the canonical design system for this repository.

It provides:
- Design tokens and theme mappings.
- Accessible React UI primitives and composed components.
- Layout, content, responsive, and interaction patterns.
- Storybook documentation and visual regression coverage.
- A controlled mapping between Figma components/variables and production code.

The goal is not merely visual consistency. The system must enable fast product delive
without sacrificing accessibility, usability, performance, or maintainability.

## Required reading

Before non-trivial UI work, read:
- `docs/design-principles.md`
- `docs/accessibility.md`
- `docs/component-inventory.md`
- The relevant file in `docs/patterns/`
- The component's existing Storybook stories, if modifying a component.

For Figma-derived work, also read:
- `docs/figma-mapping.md`

## Core rules

1. Reuse before creating.
   - Search `@semester/ui` and the component inventory before writing custom UI.
   - Prefer composing existing primitives over creating feature-local components.
   - Do not add a component when a documented composition satisfies the need.

2. Tokens are mandatory.
   - Do not add arbitrary color, spacing, radius, shadow, z-index, font-size,
     font-weight, line-height, duration, or easing values in application UI.
   - Use semantic tokens in application and component code.
   - Raw palette/reference tokens may only be used in token definitions and theme map
   - Do not edit generated token output in `dist/`.
```

```
3. Accessibility is a release requirement.
   - Target WCAG 2.2 AA.
   - Prefer native HTML elements and controls.
   - Every keyboard-reachable element must have visible focus.
   - Every icon-only control needs an accessible name.
   - Do not communicate state or validation through color alone.
   - Use semantic landmarks and heading hierarchy.
   - Dialogs must manage focus, restore it on close, and support Escape when safe.
   - Respect `prefers-reduced-motion`.
   - Maintain at least 44 by 44 CSS pixel targets for touch interactions where pract

4. Responsive behavior is intentional.
   - Implement narrow layouts first, then enhance.
   - Do not hide critical user actions on small screens.
   - Use layout primitives, container queries, and documented breakpoints.
   - State the narrow-screen strategy for each new complex view.

5. Interface quality.
   - One primary action per decision scope.
   - Use direct sentence-case labels and useful empty/error states.
   - Avoid generic dashboard decoration: arbitrary cards, excessive pills,
     gradients, glass effects, emoji as icons, or ornamental color blocks.
   - Favor clear hierarchy, scannability, and appropriate information density.
   - Use `@semester/icons`; never use emoji or ad-hoc SVGs as UI icons.

6. Dependencies and architecture.
   - Do not introduce a UI dependency without explicit approval.
   - Do not modify public component APIs without considering migration impact.
   - Do not modify generated files manually.
   - Do not disable lint, type, test, or accessibility checks to complete a task.

## Required UI workflow

For a new or substantially revised UI flow:

1. Inspect the nearest equivalent implementation and existing Storybook stories.
2. Identify existing Semester components, patterns, and semantic tokens to reuse.
3. For a complex flow, present a short plan before implementation:
   - Existing components to reuse.
   - New composition or primitive needed, if any.
   - Responsive approach.
   - Loading, empty, error, disabled, and permission states.
   - Accessibility risks and mitigations.
4. Implement using `@semester/ui`, `@semester/tokens`, and `@semester/icons`.
5. Verify all meaningful states and keyboard behavior.
6. Run the relevant validation commands.
7. Summarize:
   - Files changed.
   - System components and tokens reused.
   - Accessibility behavior checked.
   - Any newly discovered design-system gap.

## Component API conventions

- Components use PascalCase; props use camelCase.
```

```
- Use `variant`, `size`, and `tone` only for documented visual choices.
- Prefer composition and slots over large boolean-prop APIs.
- Prefer explicit names such as `isLoading`, `isInvalid`, and `aria-label`.
- Defaults must be safe, accessible, and visually standard.
- Expose stable semantic subparts with `data-slot` only when consumers need styling h
- Every control must document its disabled, loading, invalid, and read-only behavior
- New components require TypeScript types, tests, Storybook stories, and accessibili

## Figma protocol

When a task references a Figma URL, frame, component, variable collection, or design
1. Use Figma MCP to inspect the selected node and its hierarchy.
2. Treat Figma as evidence of design intent, not as literal markup to reproduce.
3. Map Figma variables to existing semantic tokens before adding a token.
4. Map Figma components to existing React components before creating a component.
5. Preserve behavior, semantics, responsiveness, and accessibility even where Figma d
6. Record a mapping mismatch in `docs/figma-mapping.md` or an ADR; do not silently fo

## Design-system completion criteria

A UI task is not complete until:
- It uses documented components and semantic tokens.
- It covers required async, empty, error, disabled, and restricted states.
- It works at narrow and wide viewport sizes.
- It is keyboard-operable and has visible focus behavior.
- It has correct semantic HTML and accessible names.
- Typecheck, lint, tests, and relevant Storybook/visual checks pass.
- System documentation is updated if the source of truth changes.

## Commands

Use package scripts; do not guess command names.

Read `package.json` first, then run the appropriate commands for touched packages:
- `pnpm lint`
- `pnpm typecheck`
- `pnpm test`
- `pnpm test:visual`
- `pnpm storybook`
- `pnpm tokens:build`
- `pnpm tokens:validate`
- `pnpm design-system:audit`

## Documentation references

@docs/design-principles.md
@docs/accessibility.md
@docs/component-inventory.md
@docs/figma-mapping.md
```

Keep the `@docs/...` imports only if those files exist. The overall principle is correct even if you remove the import lines: persistent instructions should point Claude to concise, authoritative sources rather than duplicating every rule in a huge root file.

Semantic token schema

Naming model

Use a dot-delimited name that encodes **meaning**, not a particular visual value:

```
{category}.{role}.{emphasis-or-state}
```

Examples:

```
color.text.primary
color.text.muted
color.surface.canvas
color.surface.raised
color.border.subtle
color.action.primary.default
color.action.primary.hover
color.status.danger.text
space.4
radius.lg
shadow.overlay
motion.duration.fast
```

Avoid names based on visual appearance in consumer code:

```
# Bad application-facing names
blue.500
gray.200
purpleButton
largeRadius
cardShadow2

# Good application-facing names
color.action.primary.default
color.border.default
radius.control
shadow.raised
```

Token tier policy

Token tier	Example	Allowed consumers	Purpose
Reference	`ref.color.blue.600`	Token/theme files only	Raw palette, scale, and numerical values
Semantic	`color.text.primary`	Components and application UI	Stable role-based interface contract

Token tier	Example	Allowed consumers	Purpose
Component	`button.primary.background.default`	Component implementation only	Component-specific alias where semantic roles need narrowing
Local	CSS custom property scoped to a component	Rarely, with review	Temporary/internal implementation detail, never a new global convention

`primitives.ts`

This contains raw values. Do not import it directly into React components.

```
// packages/tokens/src/primitives.ts
export const primitive = {
  color: {
    white: "#FFFFFF",
    black: "#0A0A0B",

    neutral: {
      0: "#FFFFFF",
      25: "#FCFCFD",
      50: "#F8F9FA",
      100: "#F1F3F5",
      200: "#E9ECEF",
      300: "#DEE2E6",
      400: "#CED4DA",
      500: "#ADB5BD",
      600: "#6C757D",
      700: "#495057",
      800: "#343A40",
      900: "#212529",
      950: "#101214",
    },

    brand: {
      50: "#EEF4FF",
      100: "#D9E7FF",
      200: "#BBD7FF",
      300: "#8CC0FF",
      400: "#55A0FF",
      500: "#2F7CF6",
      600: "#1E63D8",
      700: "#194FAC",
      800: "#1B438C",
      900: "#1B396F",
    },

    red: {
      50: "#FEF2F2",
      100: "#FEE2E2",
      200: "#FECACA",
      500: "#EF4444",
```

```
      600: "#DC2626",
      700: "#B91C1C",
      900: "#7F1D1D",
    },

    amber: {
      50: "#FFFBEB",
      100: "#FEF3C7",
      500: "#F59E0B",
      600: "#D97706",
      700: "#B45309",
      900: "#78350F",
    },

    green: {
      50: "#F0FDF4",
      100: "#DCFCE7",
      500: "#22C55E",
      600: "#16A34A",
      700: "#15803D",
      900: "#14532D",
    },
  },

  space: {
    0: "0",
    px: "1px",
    0_5: "0.125rem",
    1: "0.25rem",
    1_5: "0.375rem",
    2: "0.5rem",
    2_5: "0.625rem",
    3: "0.75rem",
    4: "1rem",
    5: "1.25rem",
    6: "1.5rem",
    8: "2rem",
    10: "2.5rem",
    12: "3rem",
    16: "4rem",
    20: "5rem",
    24: "6rem",
  },

  radius: {
    none: "0",
    xs: "0.125rem",
    sm: "0.25rem",
    md: "0.375rem",
    lg: "0.5rem",
    xl: "0.75rem",
    "2xl": "1rem",
    full: "9999px",
  },

  font: {
```

```
    family: {
      sans: "Inter, ui-sans-serif, system-ui, sans-serif",
      mono: '"JetBrains Mono", ui-monospace, SFMono-Regular, monospace',
    },
    size: {
      xs: "0.75rem",
      sm: "0.875rem",
      md: "1rem",
      lg: "1.125rem",
      xl: "1.25rem",
      "2xl": "1.5rem",
      "3xl": "1.875rem",
      "4xl": "2.25rem",
    },
    weight: {
      regular: 400,
      medium: 500,
      semibold: 600,
      bold: 700,
    },
    lineHeight: {
      tight: 1.15,
      snug: 1.3,
      normal: 1.5,
      relaxed: 1.65,
    },
  },

  duration: {
    instant: "0ms",
    fast: "120ms",
    normal: "180ms",
    slow: "280ms",
  },

  easing: {
    standard: "cubic-bezier(0.2, 0, 0, 1)",
    enter: "cubic-bezier(0, 0, 0.2, 1)",
    exit: "cubic-bezier(0.4, 0, 1, 1)",
  },

  zIndex: {
    base: 0,
    raised: 10,
    sticky: 100,
    dropdown: 200,
    popover: 300,
    overlay: 400,
    modal: 500,
    toast: 600,
    tooltip: 700,
  },
} as const;
```

`semantic.ts`

These are the names React components should consume. The example uses token aliasing. If you use Style Dictionary, Tokens Studio, or another token pipeline, represent the same relationships in the appropriate source format.

```
// packages/tokens/src/semantic.ts
export const semantic = {
  color: {
    text: {
      primary: "{ref.color.neutral.950}",
      secondary: "{ref.color.neutral.700}",
      muted: "{ref.color.neutral.600}",
      disabled: "{ref.color.neutral.500}",
      inverse: "{ref.color.neutral.0}",
      link: "{ref.color.brand.700}",
      linkHover: "{ref.color.brand.800}",
      danger: "{ref.color.red.700}",
      success: "{ref.color.green.700}",
      warning: "{ref.color.amber.700}",
    },

    icon: {
      primary: "{color.text.primary}",
      secondary: "{color.text.secondary}",
      muted: "{color.text.muted}",
      disabled: "{color.text.disabled}",
      inverse: "{color.text.inverse}",
      brand: "{color.action.primary.default}",
      danger: "{color.status.danger.strong}",
    },

    surface: {
      canvas: "{ref.color.neutral.25}",
      base: "{ref.color.neutral.0}",
      raised: "{ref.color.neutral.0}",
      sunken: "{ref.color.neutral.50}",
      overlay: "{ref.color.neutral.0}",
      inverse: "{ref.color.neutral.950}",
      selected: "{ref.color.brand.50}",
      disabled: "{ref.color.neutral.100}",
    },

    border: {
      subtle: "{ref.color.neutral.200}",
      default: "{ref.color.neutral.300}",
      strong: "{ref.color.neutral.400}",
      focus: "{ref.color.brand.500}",
      disabled: "{ref.color.neutral.200}",
      danger: "{ref.color.red.500}",
    },

    action: {
      primary: {
        default: "{ref.color.brand.600}",
```

```
      hover: "{ref.color.brand.700}",
      active: "{ref.color.brand.800}",
      foreground: "{ref.color.white}",
      disabled: "{ref.color.neutral.300}",
      disabledForeground: "{ref.color.neutral.600}",
    },

    secondary: {
      default: "{color.surface.base}",
      hover: "{ref.color.neutral.50}",
      active: "{ref.color.neutral.100}",
      foreground: "{color.text.primary}",
      border: "{color.border.default}",
    },

    subtle: {
      default: "transparent",
      hover: "{ref.color.neutral.100}",
      active: "{ref.color.neutral.200}",
      foreground: "{color.text.primary}",
    },

    danger: {
      default: "{ref.color.red.600}",
      hover: "{ref.color.red.700}",
      active: "{ref.color.red.900}",
      foreground: "{ref.color.white}",
    },
  },

  status: {
    info: {
      subtle: "{ref.color.brand.50}",
      default: "{ref.color.brand.600}",
      strong: "{ref.color.brand.700}",
      text: "{ref.color.brand.800}",
    },
    success: {
      subtle: "{ref.color.green.50}",
      default: "{ref.color.green.600}",
      strong: "{ref.color.green.700}",
      text: "{ref.color.green.900}",
    },
    warning: {
      subtle: "{ref.color.amber.50}",
      default: "{ref.color.amber.600}",
      strong: "{ref.color.amber.700}",
      text: "{ref.color.amber.900}",
    },
    danger: {
      subtle: "{ref.color.red.50}",
      default: "{ref.color.red.600}",
      strong: "{ref.color.red.700}",
      text: "{ref.color.red.900}",
    },
  },
```

```
    focus: {
      ring: "{ref.color.brand.500}",
      ringOffset: "{color.surface.base}",
    },
  },

  typography: {
    body: {
      sm: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.sm}",
        fontWeight: "{ref.font.weight.regular}",
        lineHeight: "{ref.font.lineHeight.normal}",
      },
      md: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.md}",
        fontWeight: "{ref.font.weight.regular}",
        lineHeight: "{ref.font.lineHeight.normal}",
      },
    },

    label: {
      sm: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.sm}",
        fontWeight: "{ref.font.weight.medium}",
        lineHeight: "{ref.font.lineHeight.snug}",
      },
      md: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.md}",
        fontWeight: "{ref.font.weight.medium}",
        lineHeight: "{ref.font.lineHeight.snug}",
      },
    },

    heading: {
      sm: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.lg}",
        fontWeight: "{ref.font.weight.semibold}",
        lineHeight: "{ref.font.lineHeight.tight}",
      },
      md: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.xl}",
        fontWeight: "{ref.font.weight.semibold}",
        lineHeight: "{ref.font.lineHeight.tight}",
      },
      lg: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.2xl}",
        fontWeight: "{ref.font.weight.semibold}",
        lineHeight: "{ref.font.lineHeight.tight}",
```

```
      },
    },
  },

  space: {
    inline: {
      xs: "{ref.space.1}",
      sm: "{ref.space.2}",
      md: "{ref.space.3}",
      lg: "{ref.space.4}",
    },
    stack: {
      xs: "{ref.space.2}",
      sm: "{ref.space.3}",
      md: "{ref.space.4}",
      lg: "{ref.space.6}",
      xl: "{ref.space.8}",
      "2xl": "{ref.space.12}",
    },
    inset: {
      xs: "{ref.space.2}",
      sm: "{ref.space.3}",
      md: "{ref.space.4}",
      lg: "{ref.space.6}",
    },
    page: {
      mobile: "{ref.space.4}",
      tablet: "{ref.space.6}",
      desktop: "{ref.space.8}",
    },
  },

  size: {
    control: {
      sm: "2rem",
      md: "2.5rem",
      lg: "3rem",
    },
    icon: {
      xs: "0.875rem",
      sm: "1rem",
      md: "1.25rem",
      lg: "1.5rem",
    },
    target: {
      minimum: "2.75rem",
    },
    content: {
      readable: "42rem",
      default: "72rem",
      wide: "90rem",
    },
  },

  radius: {
    control: "{ref.radius.md}",
```

```
    surface: "{ref.radius.lg}",
    overlay: "{ref.radius.xl}",
    pill: "{ref.radius.full}",
  },

  border: {
    default: "1px",
    strong: "2px",
  },

  shadow: {
    raised: "0 1px 2px rgb(16 18 20 / 0.06), 0 1px 3px rgb(16 18 20 / 0.10)",
    overlay: "0 12px 32px rgb(16 18 20 / 0.16), 0 2px 8px rgb(16 18 20 / 0.08)",
  },

  motion: {
    duration: {
      instant: "{ref.duration.instant}",
      fast: "{ref.duration.fast}",
      normal: "{ref.duration.normal}",
      slow: "{ref.duration.slow}",
    },
    easing: {
      standard: "{ref.easing.standard}",
      enter: "{ref.easing.enter}",
      exit: "{ref.easing.exit}",
    },
  },

  zIndex: {
    base: "{ref.zIndex.base}",
    sticky: "{ref.zIndex.sticky}",
    dropdown: "{ref.zIndex.dropdown}",
    popover: "{ref.zIndex.popover}",
    overlay: "{ref.zIndex.overlay}",
    modal: "{ref.zIndex.modal}",
    toast: "{ref.zIndex.toast}",
    tooltip: "{ref.zIndex.tooltip}",
  },
} as const;
```

Component-token example

Use component tokens when a component needs a stable contract that maps to semantic intent but should not expose every internal choice to product teams.

```
// packages/tokens/src/components.ts
export const component = {
  button: {
    primary: {
      background: {
        default: "{color.action.primary.default}",
        hover: "{color.action.primary.hover}",
        active: "{color.action.primary.active}",
```

```
        disabled: "{color.action.primary.disabled}",
      },
      foreground: {
        default: "{color.action.primary.foreground}",
        disabled: "{color.action.primary.disabledForeground}",
      },
      focusRing: "{color.focus.ring}",
      height: {
        sm: "{size.control.sm}",
        md: "{size.control.md}",
        lg: "{size.control.lg}",
      },
      radius: "{radius.control}",
      gap: "{space.inline.sm}",
    },
  },

  field: {
    background: {
      default: "{color.surface.base}",
      disabled: "{color.surface.disabled}",
    },
    border: {
      default: "{color.border.default}",
      hover: "{color.border.strong}",
      focus: "{color.border.focus}",
      invalid: "{color.border.danger}",
      disabled: "{color.border.disabled}",
    },
    text: {
      value: "{color.text.primary}",
      placeholder: "{color.text.muted}",
      disabled: "{color.text.disabled}",
    },
    height: {
      sm: "{size.control.sm}",
      md: "{size.control.md}",
      lg: "{size.control.lg}",
    },
    radius: "{radius.control}",
  },
} as const;
```

Required token validation

Add a script that fails CI if:

- An application package imports `primitive` tokens.
- CSS/TSX contains raw hex, `rgb()`, `hsl()`, arbitrary shadows, or undocumented z-index values.
- A semantic token points directly to a component token, creating reverse dependency.
- Token aliases reference missing keys or circular references.

- A token change creates insufficient contrast for documented text/control combinations.
- Generated CSS, JSON, TypeScript, and Figma token exports differ from source.

Figma MCP setup

Recommended plugin installation

Figma recommends its official Claude Code plugin because it includes MCP server settings and agent skills for common workflows. Run this in your terminal:

```
claude plugin install figma@claude-plugins-official
```

Then restart Claude Code, run `/plugin`, open the **Installed** tab, select the Figma plugin, and complete the authorization flow. [16]

Confirm the connection in Claude Code:

```
/mcp
```

Manual remote setup

If you prefer manual configuration, Figma documents its hosted remote MCP endpoint as:

```
claude mcp add --transport http figma https://mcp.figma.com/mcp
```

Then restart Claude Code and authenticate when prompted. Figma identifies its remote server as the preferred option; it is hosted at `https://mcp.figma.com/mcp`. [17] [20]

Inspect your configuration with:

```
claude mcp list
claude mcp get figma
```

Manual desktop setup

The desktop server is useful when you are actively working in an open Figma file:

1. Install and open Figma Desktop.
2. Open a Figma Design file.
3. Switch to Dev Mode with nothing selected.
4. Enable the MCP server in the right sidebar.
5. Add it to Claude Code:

```
claude mcp add --transport http figma-desktop http://127.0.0.1:3845/mcp
```

6. Restart Claude Code and use `/mcp` to verify it is available. [16]

Project-local `.mcp.json`

Use this only if your team wants the connection declared alongside the repository. Do not commit secrets or static access tokens.

```
{
  "mcpServers": {
    "figma": {
      "type": "http",
      "url": "https://mcp.figma.com/mcp"
    }
  }
}
```

Prefer the official plugin when possible; it reduces configuration drift and gives the team the same Figma-aware workflow baseline. Figma MCP can surface design variables, components, layout context, and Code Connect mappings, which is exactly the context Claude needs to translate approved designs into your existing system rather than inventing new components. [16]

Figma-to-code mapping file

Create `docs/figma-mapping.md` and maintain it as an explicit contract.

```
# Semester Figma-to-Code Mapping

## Source of truth

- Code is the source of truth for component behavior, accessibility, and public APIs
- Figma is the source of truth for approved visual design and design exploration.
- Token source files in `packages/tokens/src/` are the source of truth for productio
- Figma variables are synchronized exports of the token system where possible.

## Variable mapping

| Figma collection | Figma variable | Code token | Notes |
|---|---|---|---|
| Semester / Semantic / Color | `Text / Primary` | `color.text.primary` | Default rea
| Semester / Semantic / Color | `Surface / Base` | `color.surface.base` | Standard co
| Semester / Semantic / Color | `Action / Primary / Default` | `color.action.primary
| Semester / Semantic / Space | `Stack / MD` | `space.stack.md` | Default vertical gr
| Semester / Component / Button | `Primary / Background / Hover` | `button.primary.ba

## Component mapping

| Figma component | React export | Status | Notes |
|---|---|---|---|
```

```
| `Button` | `@semester/ui/Button` | Stable | Figma variants map to `variant`, `size
| `Text field` | `@semester/ui/TextField` | Stable | Always use with `Field` when lab
| `Dialog` | `@semester/ui/Dialog` | Stable | Code owns focus management and Escape b
| `Data table` | `@semester/ui/DataTable` | Beta | Use only in approved data-heavy ro
| `Metric card` | No direct export | Deprecated | Compose `Surface`, `Stack`, `Text`

## Mapping rules

- A Figma component is not automatically a React component.
- Figma variants map only to documented public props.
- Local Figma styles must not become production tokens without review.
- New Figma variables require a code-token proposal before use in product UI.
- Record unresolved mismatches in an ADR or GitHub issue.
```

Design-sync audit

Use this mapping to audit drift between Figma and React:

Audit area	Verify	Typical failure	Fix
Token parity	Every approved Figma semantic variable has a code equivalent	Figma-only colors or spacing	Add or map an approved semantic token
Component parity	Figma variants map to a React component prop/state	Designers create a variant unavailable in code	Add documented API or simplify the Figma variant
State parity	Default, hover, focus, active, disabled, loading, error are represented	Figma shows only default/hover	Code remains accessible; document missing design state
Layout parity	Page gutters, grids, breakpoint behavior agree	Desktop frame treated as pixel-perfect universal layout	Encode responsive behavior in layout primitives
Content parity	Labels, empty copy, errors, and truncation have rules	Generic lorem or visually short sample content	Test realistic and localized strings
Accessibility parity	Semantic HTML and focus behavior exist beyond Figma	Clickable `div`, icon buttons without names	Use correct controls and accessible labels
Code Connect parity	Figma components reference real code components where supported	Figma library drifts into lookalikes	Link/maintain Code Connect references

Use this as a repeatable audit prompt in Claude Code:

```
Audit the supplied Figma frame against Semester.

1. Inspect the Figma hierarchy, variables, component instances, and layout.
2. Inspect the corresponding React route and all Semester components it uses.
3. Produce a table with:
   - Figma node or variable
   - Closest code token/component
   - Match status: match, partial, missing, obsolete
   - Severity: blocker, major, minor
   - Recommended resolution
4. Check typography, spacing, color, border, radius, elevation, responsive layout,
   interaction states, keyboard behavior, labels, empty/loading/error states,
```

```
   and component API consistency.
5. Do not make changes until the audit is complete.
6. Prefer mapping to existing Semester concepts over creating new tokens/components.
```

UI generation skill

Create `.claude/skills/build-ui/SKILL.md`.

Skills are self-contained folders whose required `SKILL.md` uses YAML frontmatter; Claude loads the skill when it becomes relevant and may load supporting references only as needed. Anthropic recommends that MCP-using skills reference tools with fully qualified `ServerName:tool_name` syntax to avoid ambiguity. [19] [21]

```
---
name: build-semester-ui
description: Build or revise React interfaces using the Semester design system. Use t
version: 1.0.0
---

# Build Semester UI

## Goal

Create production-ready React interfaces that are visibly and behaviorally native to

Prioritize:
1. Existing system reuse.
2. Semantic token correctness.
3. Accessible behavior.
4. Responsive resilience.
5. State completeness.
6. Minimal new abstractions.

## Inputs to inspect

Before writing code, inspect:

- Root `CLAUDE.md`.
- `docs/component-inventory.md`.
- Relevant files in `docs/patterns/`.
- Existing comparable screens and Storybook stories.
- `docs/figma-mapping.md` if the work includes Figma.
- The nearest package-level instructions, if present.

If a Figma URL, node, or selected Figma design is supplied:
- Inspect it using the installed Figma MCP tools.
- Use fully qualified Figma MCP tool names where required by the local MCP configura
- Identify its components, variants, variables, layout rules, and content hierarchy.
- Do not treat a static Figma frame as a complete interaction specification.

## Mandatory planning format

For a new screen, complex component, or substantial redesign, provide this concise pl
```

```
| Concern | Decision |
|---|---|
| Existing components | List components from `@semester/ui` to reuse |
| Pattern | Name the documented product pattern, or say no pattern exists |
| Tokens | List semantic token groups likely to be used |
| Responsive strategy | Explain narrow, intermediate, and wide behavior |
| States | List loading, empty, error, disabled, permissions, and success states |
| Accessibility | Identify semantic structure, focus, label, and keyboard requiremen
| System gap | State `none` or describe the smallest missing primitive/pattern |

Do not create code before identifying whether an existing component or composition wo

## Implementation rules

### Components

- Import from `@semester/ui`, `@semester/tokens`, and `@semester/icons`.
- Search the component inventory before creating any new primitive.
- Prefer composition of `Stack`, `Inline`, `Surface`, `Text`, `Button`, `Field`,
  and domain components over feature-local styling.
- Do not use generic `div` click handlers when a native interactive element is availa
- Do not create `Card`, `Panel`, `Widget`, or `Box` wrappers solely for decoration.
- Do not use emoji or arbitrary SVGs for UI icons.

### Tokens and style

- Use semantic tokens only in product UI.
- Do not use literal hex/rgb/hsl values, arbitrary pixel spacing, raw shadow strings
  arbitrary z-indexes, or undocumented animation values.
- Do not consume primitive tokens outside token/theme packages.
- Use layout primitives and semantic spacing roles rather than one-off margins.
- Use component tokens inside component implementations only.

### Responsive design

- Build narrow layouts first.
- Preserve primary actions and critical information on narrow screens.
- Use documented breakpoints or container queries.
- Tables may scroll horizontally only when no responsible card/list transformation ex
- Do not rely on hover as the only way to expose important actions.

### Accessibility

- Prefer semantic HTML and native inputs/buttons/links.
- Use visible focus styles from Semester tokens.
- Give every icon-only button an accessible name.
- Associate labels, descriptions, and validation errors with fields.
- Use `aria-live` only for relevant dynamic updates.
- Do not use color as the sole signal.
- Manage focus in dialogs, menus, comboboxes, and other composite widgets.
- Respect reduced-motion preferences.

### States

For each meaningful data-dependent surface, implement or deliberately address:
- Loading.
```

```
- Empty.
- Error and retry.
- Disabled or unavailable action.
- Success/confirmation where the action is consequential.
- Permission-restricted state when access can be denied.
- Long content, truncation, and localization expansion where relevant.

## Figma implementation protocol

When implementing from Figma:

1. Extract intent:
   - Page hierarchy.
   - Components and variants.
   - Semantic variables.
   - Layout constraints.
   - Visible states.
2. Map intent:
   - Figma variable → existing Semester semantic token.
   - Figma component → existing React component.
   - Figma layout → Semester layout primitive.
3. Resolve mismatches:
   - Reuse an existing code behavior when Figma is incomplete.
   - Do not introduce a raw value to force pixel parity.
   - Do not create a new token/component without identifying the reusable need.
4. Implement:
   - Maintain semantic HTML, keyboard interaction, state coverage, and responsive beh
5. Document:
   - Note any approved new mapping in `docs/figma-mapping.md`.

## New component threshold

A new component is justified only when all are true:

- No existing component or documented composition meets the need.
- The need appears in at least two product contexts or is clearly foundational.
- Its public API, states, accessibility contract, and responsive behavior are defined
- It can be documented with stories and tested.
- It does not encode feature-specific business logic.

If the threshold is not met, compose existing primitives locally.

## Required verification

Before declaring work complete:

1. Run typecheck, lint, and relevant tests.
2. Add or update Storybook stories for new visual variants and states.
3. Test keyboard navigation and visible focus.
4. Test narrow and wide viewports.
5. Check realistic, long, and empty content.
6. Confirm no token policy violations.
7. Summarize:
   - Components reused.
   - Tokens/patterns used.
   - States covered.
```

```
   - Accessibility behavior verified.
   - Any system gap or Figma mismatch found.

## Completion format

Return:

### Implemented
- Concise summary of UI work.

### System reuse
- Components, patterns, and tokens reused.

### State coverage
- Loading, empty, error, permissions, disabled, responsive behavior.

### Accessibility
- Semantics, labels, keyboard behavior, focus behavior.

### Validation
- Commands run and result.

### Follow-up
- Only list concrete system work still needed.
```

Design-sync audit skill

Create `.claude/skills/audit-design-sync/SKILL.md` to make "stop generating generic UI" enforceable rather than aspirational.

```
---
name: audit-semester-design-sync
description: Audit Figma designs and React implementation for Semester token, compone
version: 1.0.0
---

# Audit Semester Design Sync

## Goal

Compare a Figma design, React implementation, and Semester design-system contracts.
Find design drift before changing code.

## Required sources

Inspect:
- Root `CLAUDE.md`.
- `docs/figma-mapping.md`.
- `docs/component-inventory.md`.
- Relevant source files and Storybook stories.
- Figma hierarchy, components, variables, and layout through Figma MCP when a Figma

## Audit dimensions
```

```
Evaluate:

1. Token discipline
   - Raw values in product UI.
   - Incorrect semantic-role usage.
   - Missing Figma-to-code token mapping.
   - Theme and dark-mode compatibility, if supported.

2. Component parity
   - Figma components and variants mapped to stable React exports.
   - Duplicate or feature-local equivalents.
   - Props that no longer represent design intent.

3. State completeness
   - Default, hover, focus-visible, active, selected, disabled, loading, invalid,
     empty, error, success, and permission states where relevant.

4. Layout and responsive behavior
   - Spacing, grid, hierarchy, overflow, content width, mobile transformations,
     touch targets, and action priority.

5. Accessibility
   - Semantic elements, accessible names, labels, descriptions, errors,
     focus management, keyboard interaction, contrast, and reduced motion.

6. Content and hierarchy
   - Clear primary action.
   - Realistic content length.
   - Empty/error explanations and recovery actions.
   - Sentence case and Semester terminology.

## Required output

Produce these sections before recommending code changes:

### Executive assessment
One paragraph: alignment level and highest-risk issues.

### Findings

| Severity | Figma/code location | Category | Finding | Existing Semester solution |
|---|---|---|---|---|---|
| Blocker/Major/Minor | Node, file, component | Token/component/a11y/layout/state/co

### Missing system capabilities

| Candidate capability | Repeated need? | Existing composition viable? | Recommendat
|---|---:|---:|---|
| Name | Yes/No | Yes/No | Compose, document pattern, or propose component |

### Implementation order
List fixes in dependency order: blockers, shared primitives, screen composition, pol

## Rules

- Do not recommend pixel-perfect copying when it violates Semester tokens or accessi
```

```
- Do not recommend a new component until existing components/compositions are exhaus
- Classify raw token usage as a defect, not an aesthetic preference.
- Favor the smallest correction that strengthens the system rather than a one-off ex
- Never modify code during audit mode unless explicitly instructed after the report.
```

Component-creation skill

This prevents Claude from responding to every visual difference by creating a new “Card”, “Section”, or “FancyButton.”

```
---
name: create-semester-component
description: Propose and implement a reusable Semester component only when existing c
version: 1.0.0
---

# Create Semester Component

## Gate

Do not create a new component until you have:

1. Searched `@semester/ui`, the component inventory, and relevant patterns.
2. Identified at least two legitimate use cases or a foundational platform need.
3. Written a concise proposal covering:
   - Problem and users.
   - Existing alternatives considered.
   - Proposed public API.
   - Variants and states.
   - Accessibility and keyboard contract.
   - Responsive behavior.
   - Token dependencies.
   - Migration/adoption plan.
4. Obtained approval if the repository requires it.

## Implementation requirements

A new component must include:
- TypeScript implementation and public export.
- Semantic/component token mapping.
- Unit or interaction tests.
- Storybook stories for all variants and significant states.
- Keyboard/focus behavior.
- Loading, disabled, invalid, and empty/error behavior where applicable.
- Documentation in the component inventory.
- A production usage example or documented composition.

## API rules

- Prefer composition over booleans.
- Do not expose raw CSS or arbitrary style props as the primary API.
- Avoid feature-specific business logic.
- Do not add `className` escape hatches without a documented reason.
- Keep variant axes orthogonal: `variant`, `size`, `tone`, `state`.
```

```
- Preserve backwards compatibility or document migration.

## Completion report

Return:
- Why existing options were insufficient.
- API and accessibility contract.
- Stories/tests added.
- Tokens used.
- First adoption target.
- Any Figma mapping additions.
```

Make the system enforceable

Configuration alone will not stop generic UI. Add automated constraints.

Token lint rules

Implement ESLint/stylelint/custom AST checks to block:

```
// Reject in app and component packages
color: "#2F7CF6"
padding: "13px"
boxShadow: "0 9px 20px rgba(0,0,0,.13)"
zIndex: 9999
transition: "all 0.2s ease"
```

Require:

```
// Accept: semantic roles or typed token helpers
backgroundColor: tokens.color.action.primary.default
padding: tokens.space.inset.md
boxShadow: tokens.shadow.raised
zIndex: tokens.zIndex.popover
transitionDuration: tokens.motion.duration.fast
```

Component inventory metadata

Keep a machine-readable inventory so Claude can locate existing components without relying entirely on prose:

```
// packages/ui/src/component-manifest.ts
export const componentManifest = {
  Button: {
    status: "stable",
    import: "@semester/ui/Button",
    useFor: ["primary actions", "form submission", "destructive confirmation"],
    avoidFor: ["navigation to another URL"],
    variants: ["primary", "secondary", "subtle", "danger"],
    sizes: ["sm", "md", "lg"],
    accessibility: ["native button semantics", "loading state", "visible focus"],
```

```
  },

  TextField: {
    status: "stable",
    import: "@semester/ui/TextField",
    useFor: ["single-line user input"],
    avoidFor: ["long-form content", "search with suggestions"],
    compositions: ["Field", "FormField"],
    accessibility: ["label association", "describedby support", "invalid state"],
  },

  DataTable: {
    status: "beta",
    import: "@semester/ui/DataTable",
    useFor: ["dense multi-column datasets"],
    avoidFor: ["small mobile-first collections"],
    accessibility: ["column headers", "sort status", "keyboard navigation"],
  },
} as const;
```

CI gate

A practical CI sequence:

```
pnpm tokens:validate
pnpm tokens:build
pnpm lint
pnpm typecheck
pnpm test
pnpm test:visual
pnpm design-system:audit
```

`design-system:audit` should detect at least:

- Primitive token imports outside allowed token packages.
- Raw colors and hardcoded design values.
- Components lacking Storybook coverage.
- Components missing an accessibility note or test.
- Figma-mapped components absent from the React manifest.
- Deprecated component imports.
- Unapproved public API changes.
- New UI files that bypass `@semester/ui` despite available equivalents.

Immediate rollout order

1. Add the root `CLAUDE.md`.
2. Create `docs/component-inventory.md` and `docs/figma-mapping.md`.
3. Install the Figma plugin with `claude plugin install figma@claude-plugins-official`; authorize and validate it through `/mcp`.[16]
4. Establish the reference → semantic → component token pipeline.
5. Add `build-semester-ui` and `audit-semester-design-sync` skills.
6. Migrate the most reused primitives first: text, buttons, fields, surfaces, overlays, navigation.
7. Add token linting and visual regression checks to CI.
8. Use the audit skill on the next 3–5 production screens; turn repeat audit findings into components, patterns, or stricter lint rules.

The most important operational rule is simple: **Figma describes intent; Semester code defines the reusable accessible implementation.** Claude should use Figma MCP to understand a screen, but it should use Semester tokens, components, patterns, and validation rules to ship it.

⁂

Generate a complete Claude Code design system toolkit: production CLAUDE.md templates, token schemas, Figma MCP setup, and SKILL.md rules

Computer
Audit your React component library against Figma variables with Claude Code: automated token mapping, drift detection, and CI sync report
Computer
Show me the root CLAUDE.md file contents
Show the SKILL.md code for the audit skill
How do I configure the Figma MCP server

Below is a complete, copy-ready toolkit for **Semester**: a React design system governed through Claude Code, tokens, Figma MCP, skills, and CI drift reports.

Use the root `CLAUDE.md` for lasting project rules, and use focused skills for procedural work such as auditing design drift. Claude Code loads skills from `.claude/skills/<name>/SKILL.md`; the file's YAML frontmatter controls discovery and invocation, while the body contains the workflow.[31]

Toolkit layout

```
semester/
├── CLAUDE.md
├── .mcp.json
├── .claude/
│   ├── settings.json
│   └── skills/
│       ├── build-semester-ui/
│       │   └── SKILL.md
│       ├── audit-semester-design-sync/
│       │   └── SKILL.md
│       └── create-semester-component/
│           └── SKILL.md
├── docs/
│   ├── component-inventory.md
│   ├── figma-mapping.md
│   ├── accessibility.md
│   ├── design-principles.md
│   └── patterns/
│       ├── forms.md
│       ├── navigation.md
│       ├── data-views.md
│       └── feedback.md
├── packages/
│   ├── tokens/
│   │   ├── src/
│   │   │   ├── primitives.ts
│   │   │   ├── semantic.ts
│   │   │   ├── components.ts
│   │   │   ├── themes/
│   │   │   │   ├── light.ts
│   │   │   │   └── dark.ts
│   │   │   └── index.ts
│   │   └── package.json
│   ├── ui/
│   │   ├── src/
│   │   ├── stories/
│   │   └── component-manifest.ts
│   └── eslint-config/
├── scripts/
│   ├── audit-token-usage.ts
│   ├── build-figma-token-map.ts
│   ├── detect-figma-drift.ts
│   └── design-system-report.ts
└── reports/
    └── design-system/
```

The enforced value chain should be:

```
Reference values
  → semantic tokens
    → component tokens
```

```
      → React components
        → product screens
```

Product code should almost never consume a raw palette value such as `blue.600`. It should consume a role such as `color.action.primary.default`.

Root CLAUDE.md

Save this exact file at `CLAUDE.md` in the repository root.

```
# Semester Design System

## Purpose

Semester is the canonical product design system for this repository.

It defines:
- Design tokens and theme mappings.
- Accessible React primitives and components.
- Layout, typography, responsive, interaction, and content conventions.
- Figma-to-code mappings.
- Storybook, test, lint, visual-regression, and CI quality requirements.

Build interfaces that are deliberate, calm, accessible, information-rich, and native
to Semester. Do not generate generic dashboard UI, decorative card grids, arbitrary
pills, gradient-heavy surfaces, or one-off visual values.

## Required reading

Before doing non-trivial UI work, read:

- `docs/design-principles.md`
- `docs/accessibility.md`
- `docs/component-inventory.md`
- The relevant document in `docs/patterns/`
- Existing Storybook stories for components being changed
- `docs/figma-mapping.md` when work references Figma

## Source-of-truth policy

- Code is the source of truth for component behavior, accessibility, public APIs,
  responsive behavior, and production token values.
- Figma is the source of truth for approved visual intent and design exploration.
- `packages/tokens/src/` is the source of truth for token definitions.
- Generated token artifacts must never be edited manually.
- Figma variables must map to semantic code tokens before they are used in product UI
- A Figma component is not automatically a React component.

## Non-negotiable rules

### Reuse system assets first

- Search `@semester/ui`, Storybook, `component-manifest.ts`, and the component
  inventory before writing UI.
```

```
- Prefer composing existing primitives over creating feature-local components.
- Do not create a new primitive, component, token, layout abstraction, or visual
  variant until existing options have been evaluated.
- Do not create generic names such as `Card`, `Box`, `Panel`, `Widget`, or
  `FancyButton` for feature-local wrappers.

### Use tokens only

- Use semantic tokens in React components and product application code.
- Do not import raw/reference tokens outside `packages/tokens`.
- Do not add arbitrary hex, rgb, hsl, oklch, spacing, radius, border, shadow,
  font, z-index, opacity, duration, or easing values in UI code.
- Do not use arbitrary Tailwind values such as `p-[13px]`, `bg-[#...]`,
  `z-[^3_9999]`, or `shadow-[...]`.
- Component-specific tokens are allowed only inside their owning component package.
- Do not edit generated CSS, JSON, TypeScript, or Figma-export token artifacts.

### Build accessible interfaces

- Meet WCAG 2.2 AA unless a documented exception exists.
- Prefer native HTML semantics and native controls.
- Use visible focus styles for every keyboard-reachable interactive element.
- Every icon-only control must have an accessible name.
- Associate field labels, descriptions, validation messages, and errors with fields.
- Do not communicate meaning through color alone.
- Dialogs, menus, popovers, comboboxes, and similar widgets must manage focus correc
- Restore focus after closing overlays.
- Support Escape to dismiss transient overlays when safe.
- Respect `prefers-reduced-motion`.
- Maintain a practical minimum 44 by 44 CSS pixel touch target.

### Design responsive interfaces intentionally

- Start with narrow layouts and enhance for larger viewports.
- Use Semester layout primitives, documented breakpoints, or approved container queri
- Do not hide critical actions or information on smaller screens.
- Do not rely exclusively on hover for essential actions.
- Handle content overflow, localization expansion, long labels, and long lists.
- State the narrow, intermediate, and wide behavior for complex layouts.

### Maintain product quality

- Establish clear hierarchy and one primary action per decision scope.
- Use sentence case for interface text unless a documented product term requires othe
- Use `@semester/icons`; never use emoji as product interface icons.
- Include meaningful loading, empty, error, disabled, success, and permission-restric
  states when relevant.
- Avoid unnecessary decoration. Prioritize useful content and scannability.
- Do not introduce UI dependencies without explicit approval.
- Do not weaken lint, typecheck, test, accessibility, or visual-regression checks.

## Required implementation workflow

For a new screen, new flow, or substantial UI change:

1. Inspect the closest existing implementation and relevant Storybook stories.
```

```
2. Identify the Semester components, layout primitives, patterns, and semantic tokens
3. For non-trivial work, provide a concise plan before editing:
   - Components and patterns to reuse.
   - Required states.
   - Responsive behavior.
   - Accessibility considerations.
   - Proposed design-system gap, if any.
4. Implement using `@semester/ui`, `@semester/tokens`, and `@semester/icons`.
5. Check keyboard operation, focus visibility, semantic structure, labels, and errors
6. Verify narrow and wide viewport behavior.
7. Run relevant validation commands.
8. Summarize reused system assets and any unresolved system gap.

## Component API rules

- Use PascalCase for component names and camelCase for props.
- Use `variant`, `size`, and `tone` only for documented, stable design choices.
- Prefer composition and slots over large boolean-prop APIs.
- Use explicit booleans such as `isLoading`, `isInvalid`, and `isDisabled`.
- Defaults must be safe, accessible, and visually standard.
- Do not expose arbitrary style props as a primary API.
- Use `data-slot` only for stable, documented component subparts.
- New components require TypeScript types, tests, Storybook coverage, documentation,
  and accessibility notes.

## Figma protocol

When a task includes a Figma URL, node ID, selected layer, variable, component, or fi

1. Use Figma MCP to inspect the design hierarchy, layout, variables, components,
   component properties, and visible states.
2. Treat Figma as design intent, not literal HTML/CSS to duplicate.
3. Map Figma variables to existing Semester semantic tokens before adding tokens.
4. Map Figma components to existing React components before creating components.
5. Preserve accessibility, semantics, responsive rules, error handling, and interacti
   behavior even when Figma does not show them.
6. Record important unmapped variables/components in `docs/figma-mapping.md`.
7. Do not create a Figma-driven exception that bypasses Semester.

## Completion requirements

A UI task is incomplete until:

- It uses approved Semester components and semantic tokens.
- It handles relevant loading, empty, error, disabled, success, and restricted states
- It works at narrow and wide viewport sizes.
- It is keyboard-operable with visible focus.
- It uses semantic HTML and accessible names.
- Typecheck, lint, relevant tests, and visual checks pass.
- Documentation is updated when the design-system source of truth changes.

## Commands

Inspect `package.json` before running commands. Use available scripts rather than gue

Common validation commands:
```

```bash
pnpm tokens:validate
pnpm tokens:build
pnpm lint
pnpm typecheck
pnpm test
pnpm test:visual
pnpm storybook
pnpm design-system:audit
pnpm design-system:report
```

Reference documents

@docs/design-principles.md
@docs/accessibility.md
@docs/component-inventory.md
@docs/figma-mapping.md

````
`CLAUDE.md` should contain lasting facts and guardrails. Keep procedural checks—such

## Audit SKILL.md

Create this directory:

```bash
mkdir -p .claude/skills/audit-semester-design-sync
```
````

Then save the following as `.claude/skills/audit-semester-design-sync/SKILL.md`.

```
---
name: audit-semester-design-sync
description: Audit a React screen, component library, or Figma implementation for Ser
when_to_use: Use for Figma-to-code reviews, design QA, React library audits, token ma
argument-hint: "[figma-url-or-node] [route-or-component-path]"
allowed-tools:
  - Read
  - Glob
  - Grep
  - Bash
---

# Audit Semester Design Sync

## Objective

Audit the alignment between:

1. The requested or supplied Figma design.
2. The React implementation.
3. Semester token definitions.
4. Semester component contracts.
```

```
5. Accessibility and responsive behavior requirements.

Find system drift before recommending changes. Do not modify source files unless the
explicitly asks for implementation after reviewing the audit.

## Required inputs

Read before auditing:

- Root `CLAUDE.md`.
- `docs/design-principles.md`.
- `docs/accessibility.md`.
- `docs/component-inventory.md`.
- `docs/figma-mapping.md`.
- Relevant files in `docs/patterns/`.
- Relevant source files, tests, and Storybook stories.
- `packages/tokens/src/semantic.ts`.
- `packages/tokens/src/components.ts`.
- `packages/ui/src/component-manifest.ts`, if present.

If a Figma reference is supplied:

1. Use Figma MCP to inspect the selected node, child hierarchy, components, componen
   properties, variables, layout data, text, and visible states.
2. Capture design intent, not merely dimensions or colors.
3. Identify which Figma variables and components should map to Semester tokens and Re
4. Do not treat a static desktop frame as a complete responsive or behavioral specif

## Audit process

### Step 1: Establish scope

State the audit target:

- Figma file, page, frame, node, or component.
- React route, feature, package, or component.
- Relevant Semester pattern.
- Existing component APIs and token namespaces.
- Whether the audit is visual-only, implementation-only, or Figma-to-code.

If the target cannot be determined, audit the available source and clearly label any

### Step 2: Inspect semantic-token discipline

Check for:

- Raw CSS colors, arbitrary design values, and raw palette imports.
- Incorrect semantic role selection.
- One-off CSS variables that duplicate Semester tokens.
- Hardcoded shadows, radii, z-index values, typography, motion values, and spacing.
- Figma variables that have no code token mapping.
- Code semantic tokens that have no corresponding Figma variable where parity is requ
- Themes that are incompatible with dark mode or high-contrast modes, if supported.

Classify each issue:
```

```
- `blocker`: breaks accessibility, causes a critical brand/system contradiction, or p
- `major`: creates repeatable drift, duplication, or an inconsistent user interactio
- `minor`: cosmetic or documentation-level mismatch that does not meaningfully break

### Step 3: Inspect component parity

For every visible Figma component or code component:

- Identify the closest React export from `@semester/ui`.
- Identify the closest Figma library component.
- Compare variants, sizes, tones, states, slots, and content rules.
- Detect feature-local duplicate components.
- Identify APIs that expose implementation detail instead of design intent.
- Verify whether the React component has Storybook coverage and a documented contrac

Do not recommend a new component until you confirm that:
- An existing component is insufficient.
- A documented composition is insufficient.
- The need is reusable across at least two product contexts or is foundational.

### Step 4: Inspect interaction and state coverage

For relevant surfaces, check:

- Default.
- Hover.
- Focus-visible.
- Active/pressed.
- Selected.
- Disabled.
- Read-only.
- Loading.
- Empty.
- Error and retry.
- Invalid validation.
- Success/confirmation.
- Permission-restricted state.
- Long-content and localization behavior.
- Network/offline/degraded behavior when relevant.

Treat a missing visible Figma state as incomplete design evidence, not permission to
state from accessible product code.

### Step 5: Inspect responsive and layout behavior

Check:

- Page gutters and max-widths.
- Layout primitives and composition.
- Stack/grid transformations.
- Overflow behavior.
- Action priority on narrow screens.
- Tables and data density.
- Content truncation and wrapping.
- Touch-target size.
- Fixed/sticky layer behavior.
```

```
- Z-index use and overlay layering.
- Container queries or breakpoint consistency.

### Step 6: Inspect accessibility

Check:

- Correct semantic landmarks and heading hierarchy.
- Native button, link, input, select, and form semantics.
- Accessible names for icon-only controls.
- Label, description, validation, and error associations.
- Focus order and visible focus styles.
- Dialog, menu, tooltip, popover, and combobox focus behavior.
- Keyboard interaction.
- Color contrast and non-color state signals.
- Motion reduction.
- Live-region use only where needed.

### Step 7: Produce report before fixes

Do not edit code during audit mode.

Use the exact report format below.

## Required output format

# Semester design-sync audit

## Executive assessment

Write one concise paragraph identifying:
- Overall alignment: strong, moderate, weak, or critical drift.
- The highest-risk mismatch.
- Whether the main issue is token, component, responsive, state, accessibility, or Fi

## Scope

| Area | Audited target | Evidence reviewed |
|---|---|---|
| Figma | Value or `not supplied` | Nodes, variables, components, states |
| Code | Value or `not supplied` | Files, stories, tests |
| Semester | Relevant packages/docs | Tokens, components, patterns |

## Findings

| Severity | Location | Category | Finding | Existing Semester solution | Recommended
|---|---|---|---|---|---|
| Blocker/Major/Minor | Figma node, file, component, selector, or token | Token/Compo

## Token mapping

| Figma collection/variable | Expected Semester token | Actual implementation | Statu
|---|---|---|---|---|
| Value | Value | Value | Match/Partial/Missing/Obsolete | Specific resolution |

## Component mapping
```

```
| Figma component | React component | Variant/state parity | Status | Action |
|---|---|---|---|---|
| Value | Value | Match/Partial/Missing | Match/Partial/Missing/Obsolete | Specific r

## State coverage

| Surface | Present | Missing or unclear | Required action |
|---|---|---|---|
| Value | List covered states | List missing states | Specific resolution |

## Accessibility review

| Requirement | Status | Evidence | Required action |
|---|---|---|---|
| Semantics | Pass/Partial/Fail | Specific element or component | Specific resolutio
| Keyboard | Pass/Partial/Fail | Specific interaction | Specific resolution |
| Focus | Pass/Partial/Fail | Specific state | Specific resolution |
| Labels/errors | Pass/Partial/Fail | Specific field/control | Specific resolution |
| Contrast/non-color cues | Pass/Partial/Fail | Specific token/state | Specific reso

## System gaps

| Candidate capability | Reusable need | Existing composition works | Recommendation
|---|---:|---:|---|
| Value | Yes/No | Yes/No | Compose, document pattern, propose token, or propose comp

## Recommended implementation order

1. List blockers first.
2. Then shared token/component issues.
3. Then route-level composition issues.
4. Then visual polish and documentation.

## Rules

- Prefer an existing Semester component over a Figma-specific clone.
- Prefer a semantic token over a visual-value token.
- Prefer a documented composition over a new component.
- Do not recommend raw values to chase pixel-perfect parity.
- Do not use Figma omissions to excuse missing accessibility or state behavior.
- Report evidence, not subjective aesthetic preference.
- Keep proposed fixes small, composable, and compatible with the design system.
```

Claude Code discovers project skills from `.claude/skills/<skill-name>/SKILL.md`. A `description` tells Claude when the skill applies; users can also invoke it directly as `/audit-semester-design-sync`.[31]

Semantic token schema

Use **reference**, **semantic**, and **component** layers. The source examples below use TypeScript, but the naming and alias rules apply equally to Style Dictionary, Tokens Studio JSON, CSS custom properties, or another generation pipeline.

Token policy

Layer	Example	May be consumed by	Why it exists
Reference	`ref.color.brand.600`	Token/theme definitions only	Raw palette, sizing, type, and motion scales
Semantic	`color.action.primary.default`	All components and product UI	Stable meaning independent of raw values
Component	`button.primary.background.default`	Owning component only	Internal component contract
Local	`--feature-x-offset`	Rarely and temporarily	Must not replicate a global semantic role

`packages/tokens/src/primitives.ts`

```
export const ref = {
  color: {
    neutral: {
      0: "#FFFFFF",
      25: "#FCFCFD",
      50: "#F8F9FA",
      100: "#F1F3F5",
      200: "#E9ECEF",
      300: "#DEE2E6",
      400: "#CED4DA",
      500: "#ADB5BD",
      600: "#6C757D",
      700: "#495057",
      800: "#343A40",
      900: "#212529",
      950: "#101214",
    },
    brand: {
      50: "#EEF4FF",
      100: "#D9E7FF",
      200: "#BBD7FF",
      300: "#8CC0FF",
      400: "#55A0FF",
      500: "#2F7CF6",
      600: "#1E63D8",
      700: "#194FAC",
      800: "#1B438C",
      900: "#1B396F",
    },
    red: {
      50: "#FEF2F2",
      100: "#FEE2E2",
      200: "#FECACA",
      500: "#EF4444",
      600: "#DC2626",
      700: "#B91C1C",
      900: "#7F1D1D",
    },
```

```
    amber: {
      50: "#FFFBEB",
      100: "#FEF3C7",
      500: "#F59E0B",
      600: "#D97706",
      700: "#B45309",
      900: "#78350F",
    },
    green: {
      50: "#F0FDF4",
      100: "#DCFCE7",
      500: "#22C55E",
      600: "#16A34A",
      700: "#15803D",
      900: "#14532D",
    },
  },

  space: {
    0: "0",
    px: "1px",
    0_5: "0.125rem",
    1: "0.25rem",
    1_5: "0.375rem",
    2: "0.5rem",
    3: "0.75rem",
    4: "1rem",
    5: "1.25rem",
    6: "1.5rem",
    8: "2rem",
    10: "2.5rem",
    12: "3rem",
    16: "4rem",
    20: "5rem",
    24: "6rem",
  },

  radius: {
    none: "0",
    xs: "0.125rem",
    sm: "0.25rem",
    md: "0.375rem",
    lg: "0.5rem",
    xl: "0.75rem",
    "2xl": "1rem",
    full: "9999px",
  },

  font: {
    family: {
      sans: "Inter, ui-sans-serif, system-ui, sans-serif",
      mono: '"JetBrains Mono", ui-monospace, SFMono-Regular, monospace',
    },
    size: {
      xs: "0.75rem",
      sm: "0.875rem",
```

```
      md: "1rem",
      lg: "1.125rem",
      xl: "1.25rem",
      "2xl": "1.5rem",
      "3xl": "1.875rem",
      "4xl": "2.25rem",
    },
    weight: {
      regular: 400,
      medium: 500,
      semibold: 600,
      bold: 700,
    },
    lineHeight: {
      tight: 1.15,
      snug: 1.3,
      normal: 1.5,
      relaxed: 1.65,
    },
  },

  duration: {
    instant: "0ms",
    fast: "120ms",
    normal: "180ms",
    slow: "280ms",
  },

  easing: {
    standard: "cubic-bezier(0.2, 0, 0, 1)",
    enter: "cubic-bezier(0, 0, 0.2, 1)",
    exit: "cubic-bezier(0.4, 0, 1, 1)",
  },

  zIndex: {
    base: 0,
    sticky: 100,
    dropdown: 200,
    popover: 300,
    overlay: 400,
    modal: 500,
    toast: 600,
    tooltip: 700,
  },
} as const;
```

packages/tokens/src/semantic.ts

```
export const semantic = {
  color: {
    text: {
      primary: "{ref.color.neutral.950}",
      secondary: "{ref.color.neutral.700}",
      muted: "{ref.color.neutral.600}",
      disabled: "{ref.color.neutral.500}",
```

```
    inverse: "{ref.color.neutral.0}",
    link: "{ref.color.brand.700}",
    linkHover: "{ref.color.brand.800}",
    danger: "{ref.color.red.700}",
    success: "{ref.color.green.700}",
    warning: "{ref.color.amber.700}",
  },

  icon: {
    primary: "{color.text.primary}",
    secondary: "{color.text.secondary}",
    muted: "{color.text.muted}",
    disabled: "{color.text.disabled}",
    inverse: "{color.text.inverse}",
    brand: "{color.action.primary.default}",
    danger: "{color.status.danger.strong}",
  },

  surface: {
    canvas: "{ref.color.neutral.25}",
    base: "{ref.color.neutral.0}",
    raised: "{ref.color.neutral.0}",
    sunken: "{ref.color.neutral.50}",
    overlay: "{ref.color.neutral.0}",
    inverse: "{ref.color.neutral.950}",
    selected: "{ref.color.brand.50}",
    disabled: "{ref.color.neutral.100}",
  },

  border: {
    subtle: "{ref.color.neutral.200}",
    default: "{ref.color.neutral.300}",
    strong: "{ref.color.neutral.400}",
    focus: "{ref.color.brand.500}",
    disabled: "{ref.color.neutral.200}",
    danger: "{ref.color.red.500}",
  },

  action: {
    primary: {
      default: "{ref.color.brand.600}",
      hover: "{ref.color.brand.700}",
      active: "{ref.color.brand.800}",
      foreground: "{ref.color.neutral.0}",
      disabled: "{ref.color.neutral.300}",
      disabledForeground: "{ref.color.neutral.600}",
    },

    secondary: {
      default: "{color.surface.base}",
      hover: "{ref.color.neutral.50}",
      active: "{ref.color.neutral.100}",
      foreground: "{color.text.primary}",
      border: "{color.border.default}",
    },
```

```
      subtle: {
        default: "transparent",
        hover: "{ref.color.neutral.100}",
        active: "{ref.color.neutral.200}",
        foreground: "{color.text.primary}",
      },

      danger: {
        default: "{ref.color.red.600}",
        hover: "{ref.color.red.700}",
        active: "{ref.color.red.900}",
        foreground: "{ref.color.neutral.0}",
      },
    },

    status: {
      info: {
        subtle: "{ref.color.brand.50}",
        default: "{ref.color.brand.600}",
        strong: "{ref.color.brand.700}",
        text: "{ref.color.brand.800}",
      },
      success: {
        subtle: "{ref.color.green.50}",
        default: "{ref.color.green.600}",
        strong: "{ref.color.green.700}",
        text: "{ref.color.green.900}",
      },
      warning: {
        subtle: "{ref.color.amber.50}",
        default: "{ref.color.amber.600}",
        strong: "{ref.color.amber.700}",
        text: "{ref.color.amber.900}",
      },
      danger: {
        subtle: "{ref.color.red.50}",
        default: "{ref.color.red.600}",
        strong: "{ref.color.red.700}",
        text: "{ref.color.red.900}",
      },
    },

    focus: {
      ring: "{ref.color.brand.500}",
      ringOffset: "{color.surface.base}",
    },
  },

  space: {
    inline: {
      xs: "{ref.space.1}",
      sm: "{ref.space.2}",
      md: "{ref.space.3}",
      lg: "{ref.space.4}",
    },
    stack: {
```

```
      xs: "{ref.space.2}",
      sm: "{ref.space.3}",
      md: "{ref.space.4}",
      lg: "{ref.space.6}",
      xl: "{ref.space.8}",
      "2xl": "{ref.space.12}",
    },
    inset: {
      xs: "{ref.space.2}",
      sm: "{ref.space.3}",
      md: "{ref.space.4}",
      lg: "{ref.space.6}",
    },
    page: {
      mobile: "{ref.space.4}",
      tablet: "{ref.space.6}",
      desktop: "{ref.space.8}",
    },
  },

  typography: {
    body: {
      sm: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.sm}",
        fontWeight: "{ref.font.weight.regular}",
        lineHeight: "{ref.font.lineHeight.normal}",
      },
      md: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.md}",
        fontWeight: "{ref.font.weight.regular}",
        lineHeight: "{ref.font.lineHeight.normal}",
      },
    },

    label: {
      sm: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.sm}",
        fontWeight: "{ref.font.weight.medium}",
        lineHeight: "{ref.font.lineHeight.snug}",
      },
      md: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.md}",
        fontWeight: "{ref.font.weight.medium}",
        lineHeight: "{ref.font.lineHeight.snug}",
      },
    },

    heading: {
      sm: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.lg}",
        fontWeight: "{ref.font.weight.semibold}",
```

```
        lineHeight: "{ref.font.lineHeight.tight}",
      },
      md: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.xl}",
        fontWeight: "{ref.font.weight.semibold}",
        lineHeight: "{ref.font.lineHeight.tight}",
      },
      lg: {
        fontFamily: "{ref.font.family.sans}",
        fontSize: "{ref.font.size.2xl}",
        fontWeight: "{ref.font.weight.semibold}",
        lineHeight: "{ref.font.lineHeight.tight}",
      },
    },
  },

  size: {
    control: {
      sm: "2rem",
      md: "2.5rem",
      lg: "3rem",
    },
    icon: {
      xs: "0.875rem",
      sm: "1rem",
      md: "1.25rem",
      lg: "1.5rem",
    },
    target: {
      minimum: "2.75rem",
    },
    content: {
      readable: "42rem",
      default: "72rem",
      wide: "90rem",
    },
  },

  radius: {
    control: "{ref.radius.md}",
    surface: "{ref.radius.lg}",
    overlay: "{ref.radius.xl}",
    pill: "{ref.radius.full}",
  },

  border: {
    default: "1px",
    strong: "2px",
  },

  shadow: {
    raised: "0 1px 2px rgb(16 18 20 / 0.06), 0 1px 3px rgb(16 18 20 / 0.10)",
    overlay: "0 12px 32px rgb(16 18 20 / 0.16), 0 2px 8px rgb(16 18 20 / 0.08)",
  },
```

```
  motion: {
    duration: {
      instant: "{ref.duration.instant}",
      fast: "{ref.duration.fast}",
      normal: "{ref.duration.normal}",
      slow: "{ref.duration.slow}",
    },
    easing: {
      standard: "{ref.easing.standard}",
      enter: "{ref.easing.enter}",
      exit: "{ref.easing.exit}",
    },
  },

  zIndex: {
    base: "{ref.zIndex.base}",
    sticky: "{ref.zIndex.sticky}",
    dropdown: "{ref.zIndex.dropdown}",
    popover: "{ref.zIndex.popover}",
    overlay: "{ref.zIndex.overlay}",
    modal: "{ref.zIndex.modal}",
    toast: "{ref.zIndex.toast}",
    tooltip: "{ref.zIndex.tooltip}",
  },
} as const;
```

packages/tokens/src/components.ts

```
export const component = {
  button: {
    primary: {
      background: {
        default: "{color.action.primary.default}",
        hover: "{color.action.primary.hover}",
        active: "{color.action.primary.active}",
        disabled: "{color.action.primary.disabled}",
      },
      foreground: {
        default: "{color.action.primary.foreground}",
        disabled: "{color.action.primary.disabledForeground}",
      },
      focusRing: "{color.focus.ring}",
      height: {
        sm: "{size.control.sm}",
        md: "{size.control.md}",
        lg: "{size.control.lg}",
      },
      radius: "{radius.control}",
      gap: "{space.inline.sm}",
    },
  },

  field: {
    background: {
      default: "{color.surface.base}",
```

```
      disabled: "{color.surface.disabled}",
    },
    border: {
      default: "{color.border.default}",
      hover: "{color.border.strong}",
      focus: "{color.border.focus}",
      invalid: "{color.border.danger}",
      disabled: "{color.border.disabled}",
    },
    text: {
      value: "{color.text.primary}",
      placeholder: "{color.text.muted}",
      disabled: "{color.text.disabled}",
    },
    height: {
      sm: "{size.control.sm}",
      md: "{size.control.md}",
      lg: "{size.control.lg}",
    },
    radius: "{radius.control}",
  },

  dialog: {
    background: "{color.surface.overlay}",
    border: "{color.border.subtle}",
    radius: "{radius.overlay}",
    shadow: "{shadow.overlay}",
    overlay: "rgb(16 18 20 / 0.48)",
  },
} as const;
```

Figma MCP configuration

Figma offers remote and desktop MCP modes. Figma recommends the remote MCP server for most users because it has the broadest feature set. Its official Claude Code plugin configures the server and includes common workflow skills. [32]

Preferred: install Figma's Claude Code plugin

Run this in your terminal:

```
claude plugin install figma@claude-plugins-official
```

Then:

1. Restart Claude Code.
2. Run `/plugin`.
3. Open the **Installed** tab.
4. Select `figma`.
5. Start authorization and approve access in the browser.

6. Return to Claude Code.
7. Run `/mcp` to confirm that Figma is connected.

Figma MCP can provide structured design context from Figma files—such as components, variables, layout data, FigJam content, and Code Connect mappings—so Claude can map a selected design to your system instead of inferring it from screenshots. [32]

Manual remote configuration

Claude Code supports remote HTTP MCP servers with:

```
claude mcp add --transport http <name> <url>
```

For a shared team configuration, use `--scope project`, which writes project configuration; local scope is the default, while user scope works across your projects. [33]

For Figma's remote MCP endpoint:

```
claude mcp add --scope project --transport http figma https://mcp.figma.com/mcp
```

Restart Claude Code, then authenticate with `/mcp` if requested:

```
/mcp
```

Verify from your shell:

```
claude mcp list
claude mcp get figma
```

The status should move from authentication-required to connected after authorization. Claude Code's MCP status panel and CLI can show connected, needs-authentication, pending approval, and failed-connection states. [33]

`.mcp.json` project configuration

If you want the remote server declared in the repository, create `.mcp.json` at the project root:

```
{
  "mcpServers": {
    "figma": {
      "type": "http",
      "url": "https://mcp.figma.com/mcp"
    }
  }
}
```

The `type` field is required for URL-based JSON configuration. Claude Code treats a server without a declared `type` as a local stdio server, so an entry containing a `url` but no `type: "http"` is invalid.[33]

When teammates clone the repository:

1. They run `claude` from the project.
2. They accept the workspace trust prompt.
3. They review and approve the project MCP configuration.
4. They authenticate their own Figma account via `/mcp`.

Do not place static Figma access tokens, OAuth tokens, or secrets in `.mcp.json`. Use the OAuth authentication flow that Claude Code invokes from `/mcp`.

Desktop Figma MCP server

Use desktop MCP when your organization specifically requires it, or when you need the local Desktop workflow. Figma otherwise recommends its remote server.[32]

1. Install and open the Figma desktop app.
2. Open a Figma Design file.
3. Enter Dev Mode with no canvas layer selected.
4. Enable the MCP server in the right sidebar.
5. Copy the server URL offered by Figma.
6. Configure Claude Code:

```
claude mcp add --scope local --transport http figma-desktop http://127.0.0.1:3845/mcp
```

7. Restart Claude Code.
8. Run `/mcp` and confirm that `figma-desktop` appears.

Figma's documented local endpoint is `http://127.0.0.1:3845/mcp`.[32]

Recommended `.claude/settings.json`

This does not force secrets or automatic trust. It simply records repository-wide behavior.

```
{
  "permissions": {
    "allow": [
      "Read",
      "Glob",
      "Grep"
    ]
  }
}
```

Keep automated Figma writes disabled by default. Reading Figma variables and inspecting nodes is low-risk; changing an approved design file is a collaborative, state-changing operation that should require an explicit task and review.

Figma-to-code mapping

Create `docs/figma-mapping.md`:

```
# Semester Figma-to-Code Mapping

## Ownership

- Code owns implementation behavior, accessibility, responsive behavior, and public A
- Figma owns approved visual design intent and exploration.
- `packages/tokens/src/` owns production design-token values.
- Figma variables should represent exported/synchronized semantic token contracts.
- Figma component variants must map to documented React component props and states.

## Variable mapping

| Figma collection | Figma variable | Semester token | Status | Notes |
|---|---|---|---|---|
| Semester / Semantic / Color | Text / Primary | `color.text.primary` | Stable | Prir
| Semester / Semantic / Color | Surface / Base | `color.surface.base` | Stable | Defa
| Semester / Semantic / Color | Action / Primary / Default | `color.action.primary.de
| Semester / Semantic / Color | Border / Focus | `color.border.focus` | Stable | Focu
| Semester / Semantic / Space | Stack / MD | `space.stack.md` | Stable | Default ver
| Semester / Component / Button | Primary / Hover | `button.primary.background.hover

## Component mapping

| Figma component | React export | Figma variants → React props | Status | Notes |
|---|---|---|---|---|
| Button | `@semester/ui/Button` | `Style` → `variant`; `Size` → `size`; `Loading` →
| Text field | `@semester/ui/TextField` | `Size` → `size`; `State` → `isInvalid`, `is
| Dialog | `@semester/ui/Dialog` | `Size` → `size` | Stable | Code owns focus trap ar
| Data table | `@semester/ui/DataTable` | `Density` → `density` | Beta | Do not use
| Metric card | No direct export | N/A | Deprecated | Compose `Surface`, `Stack`, `Te

## Mapping rules

- Figma variables must map to semantic code tokens before production use.
- Figma local styles must not become code tokens without design-system review.
- Figma components are not assumed to be React components.
- Figma variants map only to documented React props.
- Missing implementation states in Figma do not remove code accessibility requiremen
- New variables or components require a documented code mapping and system review.
- Record unresolved mismatches as an ADR or tracked issue.
```

Automated token mapping and drift detection

The practical objective is a CI report that tells you whether Figma and code agree on the system contract.

Canonical mapping manifest

Create `packages/tokens/src/figma-map.ts`:

```
export type FigmaTokenMapping = {
  figmaCollection: string;
  figmaVariable: string;
  semesterToken: string;
  required: boolean;
  componentScope?: string;
};

export const figmaTokenMap: FigmaTokenMapping[] = [
  {
    figmaCollection: "Semester / Semantic / Color",
    figmaVariable: "Text / Primary",
    semesterToken: "color.text.primary",
    required: true,
  },
  {
    figmaCollection: "Semester / Semantic / Color",
    figmaVariable: "Surface / Base",
    semesterToken: "color.surface.base",
    required: true,
  },
  {
    figmaCollection: "Semester / Semantic / Color",
    figmaVariable: "Action / Primary / Default",
    semesterToken: "color.action.primary.default",
    required: true,
  },
  {
    figmaCollection: "Semester / Semantic / Space",
    figmaVariable: "Stack / MD",
    semesterToken: "space.stack.md",
    required: true,
  },
  {
    figmaCollection: "Semester / Component / Button",
    figmaVariable: "Primary / Background / Hover",
    semesterToken: "button.primary.background.hover",
    required: false,
    componentScope: "Button",
  },
];
```

Flattened-token utility

Create `scripts/flatten-tokens.ts`:

```
export type FlatTokenMap = Record<string, unknown>;

export function flattenTokens(
  value: Record<string, unknown>,
  prefix = "",
): FlatTokenMap {
  return Object.entries(value).reduce<FlatTokenMap>((result, [key, child]) => {
    const path = prefix ? `${prefix}.${key}` : key;

    if (
      child !== null &&
      typeof child === "object" &&
      !Array.isArray(child)
    ) {
      Object.assign(
        result,
        flattenTokens(child as Record<string, unknown>, path),
      );
      return result;
    }

    result[path] = child;
    return result;
  }, {});
}
```

Token-map report generator

Create `scripts/build-figma-token-map.ts`:

```
import { mkdir, writeFile } from "node:fs/promises";
import { component } from "../packages/tokens/src/components";
import { semantic } from "../packages/tokens/src/semantic";
import { figmaTokenMap } from "../packages/tokens/src/figma-map";
import { flattenTokens } from "./flatten-tokens";

type ReportStatus = "match" | "missing-code-token";

const semanticTokens = flattenTokens(semantic);
const componentTokens = flattenTokens(component);
const allTokens = {
  ...semanticTokens,
  ...componentTokens,
};

const rows = figmaTokenMap.map((mapping) => {
  const exists = Object.hasOwn(allTokens, mapping.semesterToken);

  return {
    ...mapping,
```

```
    status: (exists ? "match" : "missing-code-token") as ReportStatus,
    resolvedValue: exists ? allTokens[mapping.semesterToken] : null,
  };
});

const summary = {
  generatedAt: new Date().toISOString(),
  total: rows.length,
  matched: rows.filter((row) => row.status === "match").length,
  missingCodeTokens: rows.filter((row) => row.status === "missing-code-token")
    .length,
};

await mkdir("reports/design-system", { recursive: true });

await writeFile(
  "reports/design-system/figma-token-map.json",
  JSON.stringify({ summary, rows }, null, 2),
);

const markdown = [
  "# Semester Figma Token Map",
  "",
  `Generated: ${summary.generatedAt}`,
  "",
  `- Total mappings: ${summary.total}`,
  `- Matched: ${summary.matched}`,
  `- Missing code tokens: ${summary.missingCodeTokens}`,
  "",
  "| Figma collection | Figma variable | Semester token | Status | Resolved value |"
  "|---|---|---|---|---|",
  ...rows.map((row) => {
    const value = row.resolvedValue ?? "—";
    return `| ${row.figmaCollection} | ${row.figmaVariable} | \`${row.semesterToken}`
  }),
  "",
].join("\n");

await writeFile(
  "reports/design-system/figma-token-map.md",
  markdown,
);

if (summary.missingCodeTokens > 0) {
  process.exitCode = 1;
}
```

Raw-value and token-policy scanner

Create `scripts/audit-token-usage.ts`:

```
import { glob } from "glob";
import { readFile } from "node:fs/promises";

const files = await glob([
```

```
  "apps/**/*.{ts,tsx,js,jsx,css,scss}",
  "packages/ui/**/*.{ts,tsx,js,jsx,css,scss}",
  "!**/dist/**",
  "!**/node_modules/**",
  "!**/*.test.*",
  "!**/*.stories.*",
]);

const violations: Array<{
  file: string;
  line: number;
  rule: string;
  match: string;
}> = [];

const rules: Array<{ rule: string; expression: RegExp }> = [
  {
    rule: "raw-color",
    expression:
      /#[0-9a-fA-F]{3,8}\b|\brgba?\(|\bhsla?\(|\boklch?\(/g,
  },
  {
    rule: "arbitrary-tailwind-design-value",
    expression:
      /\b(?:bg|text|border|ring|shadow|p|px|py|pt|pr|pb|pl|m|mx|my|mt|mr|mb|ml|gap|z
  },
  {
    rule: "arbitrary-z-index",
    expression: /\bzIndex\s*:\s*\d{2,}/g,
  },
  {
    rule: "raw-shadow",
    expression: /\bboxShadow\s*:\s*["'`]/g,
  },
];

for (const file of files) {
  const content = await readFile(file, "utf8");
  const lines = content.split("\n");

  for (const [index, line] of lines.entries()) {
    for (const { rule, expression } of rules) {
      for (const match of line.matchAll(expression)) {
        violations.push({
          file,
          line: index + 1,
          rule,
          match: match[^3_0],
        });
      }
    }
  }
}

if (violations.length === 0) {
  console.log("No token-policy violations found.");
```

```
  process.exit(0);
}

console.table(violations);
process.exit(1);
```

Tune exclusions so that raw values remain allowed in `packages/tokens`, snapshots, fixtures, generated output, and intentionally isolated visual-test files.

CI report script

Add scripts like these to root `package.json`:

```
{
  "scripts": {
    "tokens:validate": "tsx packages/tokens/src/validate.ts",
    "tokens:build": "tsx scripts/build-tokens.ts",
    "design-system:map": "tsx scripts/build-figma-token-map.ts",
    "design-system:audit": "tsx scripts/audit-token-usage.ts",
    "design-system:report": "pnpm tokens:validate && pnpm design-system:map && pnpm d
  }
}
```

GitHub Actions workflow

Create `.github/workflows/design-system.yml`:

```
name: Design system CI

on:
  pull_request:
    paths:
      - "apps/**"
      - "packages/tokens/**"
      - "packages/ui/**"
      - "scripts/**"
      - "docs/figma-mapping.md"
      - ".claude/**"
      - ".github/workflows/design-system.yml"
  push:
    branches:
      - main

jobs:
  validate-design-system:
    name: Validate tokens and design-system drift
    runs-on: ubuntu-latest

    steps:
      - name: Check out repository
        uses: actions/checkout@v4
```

```
      - name: Set up pnpm
        uses: pnpm/action-setup@v4
        with:
          version: 9

      - name: Set up Node
        uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: pnpm

      - name: Install dependencies
        run: pnpm install --frozen-lockfile

      - name: Validate token aliases
        run: pnpm tokens:validate

      - name: Build token artifacts
        run: pnpm tokens:build

      - name: Build Figma-to-code token map
        run: pnpm design-system:map

      - name: Detect raw design values
        run: pnpm design-system:audit

      - name: Typecheck
        run: pnpm typecheck

      - name: Run component tests
        run: pnpm test

      - name: Run visual regression tests
        run: pnpm test:visual

      - name: Upload design-system report
        uses: actions/upload-artifact@v4
        if: always()
        with:
          name: design-system-report
          path: reports/design-system/
          if-no-files-found: warn
```

CI output contract

Have CI produce a report with this structure:

```
# Semester Design System Report

## Summary

- Token alias validation: pass
- Figma-to-code mapped variables: 84 / 88
- Missing semantic code tokens: 2
- Obsolete mappings: 1
```

```
- Raw-value violations: 3
- Unmapped Figma components: 2
- Visual-regression status: pass
- Accessibility-test status: partial

## Blockers

| Type | Location | Issue | Resolution |
|---|---|---|---|
| Raw color | `apps/web/src/billing/PlanCard.tsx:42` | `#1E63D8` bypasses semantic to
| Mapping | `Figma: Action / Positive / Default` | No matching semantic token | Propo

## Major drift

| Type | Figma | Code | Recommended action |
|---|---|---|---|
| Component | `Data table / Compact` | No supported React density mode | Add `density
| State | Error state present in Figma | No retry action in React | Add inline retry

## Token parity

| Figma variable | Semester token | Status |
|---|---|---|
| `Text / Primary` | `color.text.primary` | Match |
| `Action / Primary / Default` | `color.action.primary.default` | Match |
| `Action / Positive / Default` | — | Missing |
```

Figma variable audit workflow

For actual Figma-value inspection, use Claude Code with Figma MCP after the server is connected:

```
/audit-semester-design-sync <Figma frame URL> apps/web/src/routes/settings.tsx
```

Or prompt Claude Code directly:

```
Use the audit-semester-design-sync skill.

Inspect this Figma frame and compare it with the React implementation:
[Figma URL]

Audit:
- Figma variables against packages/tokens/src/semantic.ts.
- Figma components against @semester/ui exports.
- React source for raw colors and raw layout values.
- Figma and code state parity.
- Responsive and accessibility gaps.

Do not edit files. Produce the required design-sync audit report.
```

```
The remote HTTP MCP route is Claude Code's recommended approach for cloud MCP servers
```

Rollout sequence

1. Add the root `CLAUDE.md` and commit it.
2. Create the `audit-semester-design-sync` skill and invoke it against one existing production route.
3. Install Figma MCP using the official plugin; use the remote server by default. [32]
4. Establish reference, semantic, and component token layers.
5. Create and maintain `docs/figma-mapping.md`.
6. Add the Figma-token mapping manifest.
7. Add raw-value detection and token-alias validation to CI.
8. Audit your ten most-used components and three most important product screens.
9. Convert repeated findings into one of four outcomes: token, component, documented pattern, or lint rule.
10. Require every new Figma library variable/component to have a corresponding code mapping before it reaches production.

This gives Claude Code a constrained, inspectable contract: it can read Figma for intent, map intent to Semester primitives, detect drift automatically, and refuse to solve every UI task with another hand-styled generic component.

⁂

provide comand to claud code to excute on this and also anlyse current semester resporty and specfic to it

Your current Semester repo is already much further along than a generic “add a design system” setup. It has a dedicated token source/export path, extensive style-contract tests, an existing root `CLAUDE.md`, vendored Claude skills, and a large Vite/React application under `app/`. The best next move is **not** to replace that architecture—it is to add a Semester-specific Claude Code audit skill and tighten the existing token-export → Figma-variable → CI loop. [github_mcp_direct]

Run this in Claude Code

From the repository root, launch Claude Code:

```
cd /path/to/semester
claude
```

Then paste this command/prompt exactly into Claude Code:

```
/audit-semester-design-sync app/src app/design-tokens/semester.tokens.json app/src/s

You are auditing the existing Semester repository, not designing a generic new syste
```

```
First read:
- CLAUDE.md
- app/README.md
- DESIGN-SYSTEM-GUIDE.md
- docs/DESIGN-TOKENS.md
- docs/design/DESIGN-SYSTEM-PRODUCT-SPEC.md
- app/package.json
- app/src/styles/tokens.css
- app/src/lib/tokenexport.ts
- app/src/lib/tokenexport.test.ts
- app/design-tokens/semester.tokens.json
- app/src/styles/*.test.ts
- .claude/skills/

Then inspect:
- app/src/components/
- app/src/screens/
- app/src/styles/
- app/src/App.tsx
- app/src/main.tsx
- app/src/screens.tsx

Do not modify files yet.

Create a repository-specific Semester design-system audit with:
1. Existing architecture and source-of-truth map.
2. Token pipeline map: CSS semantic tokens → tokenexport.ts → semester.tokens.json →
3. Current design-system strengths that must be preserved.
4. Drift risks: raw styles, direct hex usage, duplicated component styles, hard-codec
5. Existing tests/scripts that already enforce design contracts.
6. Exact missing files needed for Claude Code + Figma MCP workflow only.
7. A minimal phased implementation plan that avoids restructuring the current app.
8. A table of proposed changes with priority, exact path, rationale, and acceptance t

Use evidence from actual files. Do not assume pnpm, monorepos, Storybook, Tailwind, c
```

That audit prompt deliberately tells Claude not to invent a `packages/ui`, pnpm workspace, Tailwind setup, or a replacement token schema—the patterns in earlier generic templates would be wrong for this particular repository.

One-command bootstrap prompt

Once the audit completes and you agree with its plan, paste the following command to have Claude Code implement only the smallest compatible toolkit:

```
Implement the approved Semester Claude Code design-system toolkit in this repository

Constraints:
- Preserve the current Vite + React architecture under app/.
- Preserve app/src/styles/tokens.css as the semantic token authority.
- Preserve app/src/lib/tokenexport.ts as the token export authority.
- Preserve app/design-tokens/semester.tokens.json as generated output; never hand-edi
- Preserve and extend existing tokenexport, style, accessibility, responsive, and cor
- Do not add Tailwind, Storybook, a monorepo, or a new component library package.
```

```
- Do not move existing components or rewrite App.tsx unless absolutely necessary.
- Do not add dependencies unless an existing repository dependency cannot support the
- Do not commit, push, open a PR, or change deployment configuration.

Create only:
1. .claude/skills/audit-semester-design-sync/SKILL.md
2. .claude/skills/build-semester-ui/SKILL.md
3. .mcp.json using the Figma remote MCP server, with no credentials committed
4. docs/FIGMA-MAPPING.md
5. app/scripts/design-system-audit.mjs or .ts, compatible with the app's existing too
6. app/scripts/design-system-report.mjs or .ts
7. Minimal package.json scripts in app/package.json:
   - design-system:audit
   - design-system:report
   - design-system:check
8. Focused tests that prove:
   - semester.tokens.json is generated and synchronized with code
   - raw values are not introduced outside approved CSS/token files
   - Figma mapping entries resolve to exported token paths
   - audit/report scripts fail on real violations and pass on valid fixtures

Before editing, show the file-by-file plan.
After editing, run the repository's actual relevant validation commands.
Report changed files, exact checks run, results, known limitations, and any Figma ste
```

Current Semester findings

Architecture already present

The active application is a Vite/TypeScript React app located in `app/`, not a package-per-component design-system monorepo. It has a large app entry point, screen hierarchy, component directory, styling directory, API/server code, and build configuration. [github_mcp_direct]

Current asset	What it means for your toolkit
`app/src/styles/tokens.css`	This should remain the **semantic-token authority**
`app/src/lib/tokenexport.ts`	This should remain the **token-export authority**
`app/design-tokens/semester.tokens.json`	This is the machine-readable/exported artifact; do not hand-edit it
`app/src/lib/tokenexport.test.ts`	Already validates that token export does not drift from code
`app/src/styles/*.test.ts`	You already have a substantial contract-test surface for CSS design rules
`app/src/components/` and `app/src/screens/`	These are the correct places to audit real UI composition and component duplication
`.claude/skills/`	Claude Code skill infrastructure already exists, so add one focused skill rather than bootstrapping from zero

Current asset	What it means for your toolkit
`CLAUDE.md`	Existing durable project instructions should be amended, not replaced blindly
`DESIGN-SYSTEM-GUIDE.md` and `docs/DESIGN-TOKENS.md`	Existing written design-system documentation that new Claude rules should reference

The repository's token documentation explicitly identifies `app/design-tokens/semester.tokens.json` as a generated, machine-readable export; `tokenexport.test.ts` verifies that it remains synchronized with `tokens.css`. That is a stronger foundation than a fresh generic JSON schema because it already supports downstream Figma/native/parity use cases. [github_mcp_direct]

Important constraints

Your project has a **large existing CSS system**: `app.css` is approximately 322 KB, `features.css` about 29 KB, `unity.css` about 34 KB, and `tokens.css` roughly 13 KB. It also has targeted tests for breakpoints, density, motion, fields, glass effects, gutters, hex handling, stacking, taps, typography scale, and token behavior. Treat this as a mature contract system, not a greenfield UI package. [github_mcp_direct]

So Claude Code should:

- Reuse the current CSS token vocabulary before introducing any new token name.
- Extend current source-based token export rather than create duplicate TypeScript token files.
- Use the existing style tests as enforcement hooks instead of adding broad, untested rules in a new lint stack.
- Keep the generated token JSON as the Figma/native interchange artifact.
- Add a Figma mapping manifest and a diff report around the existing export, rather than trying to make Figma the production source of truth.

Existing Figma readiness

The repository contains Figma-aware documentation and token-export comments, but the code search did **not** reveal a configured Figma MCP server or a maintained Figma-to-code component/variable mapping file. The export code and tests already describe `semester.tokens.json` as the artifact intended for Figma variables, native clients, and parity checking. [github_mcp_direct]

That means your gap is integration/governance—not token invention.

Add this audit skill

Create this folder and file:

```
mkdir -p .claude/skills/audit-semester-design-sync
```

```
.claude/skills/audit-semester-design-sync/SKILL.md
```

Paste this exact content:

```
---
name: audit-semester-design-sync
description: Audit the Semester React application against its existing CSS token sys
argument-hint: "[optional-figma-url-or-node] [optional-app-path]"
---

# Audit Semester Design Sync

## Semester-specific source of truth

Semester is an existing Vite and React application. Do not impose a generic monorepo
Tailwind, Storybook, or package-based design-system architecture unless it already ex

The source-of-truth order is:

1. `app/src/styles/tokens.css`
   - Semantic CSS token authority used by application UI.

2. `app/src/lib/tokenexport.ts`
   - Exports the application's semantic token system for downstream consumers.

3. `app/design-tokens/semester.tokens.json`
   - Generated machine-readable token artifact.
   - Never edit manually.
   - Regenerate through the repository's actual token-export command.

4. Existing CSS/component implementation
   - `app/src/styles/`
   - `app/src/components/`
   - `app/src/screens/`

5. Existing contract tests
   - `app/src/styles/*.test.ts`
   - `app/src/lib/tokenexport.test.ts`
   - Related architecture, responsive, accessibility, and screen tests.

6. Figma
   - Figma is evidence of approved visual intent.
   - Code owns production behavior, accessibility, responsive implementation, and tol
   - Figma variables/components must map to existing exported token paths and React/C

## Required reading
```

```
Before auditing, read:

- `CLAUDE.md`
- `app/README.md`
- `DESIGN-SYSTEM-GUIDE.md`
- `docs/DESIGN-TOKENS.md`
- `docs/design/DESIGN-SYSTEM-PRODUCT-SPEC.md`
- `app/package.json`
- `app/src/styles/tokens.css`
- `app/src/lib/tokenexport.ts`
- `app/src/lib/tokenexport.test.ts`
- `app/design-tokens/semester.tokens.json`
- `app/src/styles/*.test.ts`
- `docs/FIGMA-MAPPING.md`, if it exists

For a UI-specific audit, inspect the target implementation plus nearest related scree
components, and styles.

## Figma workflow

If a Figma URL, selection, node, component, or variable is supplied:

1. Use configured Figma MCP tools to inspect:
   - Node hierarchy.
   - Figma variables and collections.
   - Component instances and properties.
   - Auto-layout, constraints, and visible states.
   - Text/content hierarchy.

2. Compare Figma variable names to:
   - `app/design-tokens/semester.tokens.json`
   - `docs/FIGMA-MAPPING.md`
   - Semantic names in `app/src/styles/tokens.css`

3. Compare Figma components to current React/CSS components.
4. Do not create raw CSS to obtain pixel-perfect parity.
5. Do not treat an incomplete Figma frame as permission to omit:
   - Keyboard support.
   - Focus visibility.
   - Empty/loading/error states.
   - Responsive behavior.
   - Accessible labels and semantic HTML.

## Audit checks

### Token integrity

Identify:

- Direct hex, rgb, hsl, oklch, arbitrary shadows, arbitrary spacing, arbitrary radii
  arbitrary z-indexes, arbitrary durations, or arbitrary easing values.
- Whether each raw value is permitted in a foundational source file or is an app-leve
- CSS variables used but absent from `tokens.css`.
- CSS tokens present but not represented in the generated export.
- Generated token paths that cannot be traced back to semantic CSS tokens.
- Figma variables that lack an exported token mapping.
```

```
- Exported semantic tokens that lack an approved Figma mapping, if parity is required

### Component and pattern integrity

Identify:

- Feature-local components duplicating existing component or CSS patterns.
- Hard-coded styling that bypasses existing classes or semantic variables.
- Missing or inconsistent states.
- Misused controls: buttons vs links, clickable divs, unlabeled icon controls.
- Inconsistent density, hierarchy, type scale, surface treatment, borders, and page g
- Component APIs or CSS selectors that leak implementation detail.

### Responsive integrity

Check:

- Current breakpoint/test contracts.
- Narrow, intermediate, and wide behavior.
- Overflow, scroll, truncation, long copy, localization, and touch targets.
- Fixed/sticky element and stacking behavior.
- Data/table fallback behavior on narrow layouts.

### Accessibility integrity

Check:

- Semantic landmarks and headings.
- Label, description, error, and input associations.
- Keyboard interaction and visible focus.
- Dialog/menu/popover focus behavior.
- Color contrast and non-color status communication.
- Reduced-motion behavior.
- Target sizing.

## Severity scale

- Blocker: breaks critical usability, accessibility, core system integrity, or releas
- Major: repeatable drift, broken component/pattern contract, significant responsive,
- Minor: localized inconsistency, documentation gap, or low-risk visual mismatch.

## No-edit audit mode

Do not modify source code during an audit unless the user explicitly asks after revie
the report. You may run read-only inspection commands and existing test/audit commanc

## Required report format

# Semester design-system audit

## Executive assessment

State:
- Overall alignment: strong, moderate, weak, or critical drift.
- Highest-risk finding.
- Whether the main concern is tokens, components, responsive behavior, accessibility
```

```
  Figma parity, or test coverage.

## Repository model

| Concern | Current Semester authority | Evidence |
|---|---|---|
| Semantic tokens | `app/src/styles/tokens.css` | Specific section/file |
| Token export | `app/src/lib/tokenexport.ts` | Specific function/file |
| Machine-readable artifact | `app/design-tokens/semester.tokens.json` | Specific fi
| UI implementation | Components/screens/styles | Specific paths |
| Enforcement | Existing tests/scripts | Exact script or test paths |
| Figma parity | Mapping file or missing | Evidence |

## Findings

| Severity | Location | Category | Evidence | Existing Semester solution | Smallest s
|---|---|---|---|---|---|
| Blocker/Major/Minor | Exact path, selector, token, or Figma node | Token/Component/

## Token parity

| Figma variable or code token | Semester exported path | CSS semantic source | Statu
|---|---|---|---|---|
| Value | Value | Value | Match/Partial/Missing/Obsolete | Specific resolution |

## Component parity

| Figma or screen component | Current React/CSS implementation | State/variant parity
|---|---|---|---|---|---|
| Value | Value | Match/Partial/Missing | Match/Partial/Missing/Obsolete | Specific r

## Existing enforcement

| Contract | Existing test/script | Coverage | Gap |
|---|---|---|---|
| Token export | Exact test/script | What it proves | What remains |
| Responsive | Exact test/script | What it proves | What remains |
| Accessibility | Exact test/script | What it proves | What remains |
| Raw values | Exact test/script | What it proves | What remains |

## Minimal implementation plan

| Priority | Exact path | Change | Why | Acceptance test |
|---|---|---|---|---|
| P0/P1/P2 | Exact repo path | Smallest concrete change | System value | Exact commar

## Rules for recommendations

- Preserve Semester's existing token and export architecture.
- Recommend extensions, not parallel sources of truth.
- Use current tests before introducing tooling.
- Do not recommend Tailwind, Storybook, a monorepo, or package migrations unless
  directly supported by evidence and explicitly requested.
- Do not create a component for a one-off visual difference.
- Never recommend direct raw values to match Figma pixels.
- Prefer semantic CSS variables, existing classes, documented CSS patterns, and curre
```

Figma MCP setup

For Semester, configure the **remote** Figma MCP endpoint at the repository level, but do not commit credentials.

Run from the repository root:

```
claude mcp add --scope project --transport http figma https://mcp.figma.com/mcp
```

Then launch Claude Code:

```
claude
```

In Claude Code:

```
/mcp
```

```
Authorize Figma through the browser prompt, then verify Figma is connected. Claude Co
```

If you prefer a committed project declaration, add a root `.mcp.json`:

```
{
  "mcpServers": {
    "figma": {
      "type": "http",
      "url": "https://mcp.figma.com/mcp"
    }
  }
}
```

The `type: "http"` field matters: Claude Code needs it to interpret the URL as a remote HTTP MCP server instead of an attempted local stdio server. [34]

Figma recommends its official Claude Code plugin as the lowest-friction approach, because the plugin includes the MCP configuration and Figma-related workflow skills:

```
claude plugin install figma@claude-plugins-official
```

Use **one** setup route initially:

- Prefer the official plugin if you want Figma's maintained defaults and bundled workflows.
- Prefer the checked-in `.mcp.json` if you need a transparent, reviewable repository configuration.
- Avoid configuring duplicate servers under multiple names until you have a real reason; duplicated MCP tools make Claude's tool selection less reliable. Figma MCP gives Claude

structured access to variables, components, layout information, and other design context used for implementation and audit work. [35]

What to add—not replace

Add now	Why
`.claude/skills/audit-semester-design-sync/SKILL.md`	Gives Claude a repeatable repo-specific design review workflow
`.claude/skills/build-semester-ui/SKILL.md`	Ensures future UI work follows existing tokens, CSS, components, and test contracts
`docs/FIGMA-MAPPING.md`	Makes token/component parity explicit and reviewable
`.mcp.json` or the Figma plugin	Gives Claude structured Figma context
`app/scripts/design-system-audit.*`	Detects prohibited raw styling and missing token paths
`app/scripts/design-system-report.*`	Produces a CI-readable design-system drift summary
Package scripts in `app/package.json`	Makes the audit reproducible locally and in CI
Do not add yet	Why
---	---
A second token source in `packages/tokens/`	Semester already has `tokens.css` + `tokenexport.ts` + generated JSON
A generic `@semester/ui` package	No evidence that your current app uses package-based UI distribution
Tailwind/token migration	Would create a parallel styling system and increase drift risk
Storybook as a prerequisite	Useful later, but first use your established CSS contracts and app-screen tests
A new component framework	Existing components/screens/styles need an evidence-based audit first
Automatic Figma writes	Figma reads are appropriate for audit; design-file mutations need explicit review

The repository is public and currently identified as `harrisonjrubin7-cmyk/semester`, with `main` as the default branch. It contains roughly 53 open issues and a substantial TypeScript codebase, so start with a **read-only audit and minimal additive configuration**, rather than a sweeping UI-platform rewrite. [github_mcp_direct]

⁂

1. https://code.claude.com/docs/en/memory
2. https://code.claude.com/docs/en/skills
3. https://github.com/anthropics/claude-code
4. https://www.reddit.com/r/ClaudeAI/comments/1m6hek6/claude_project_loaded_with_all_claude_code_docs/
5. https://www.freecodecamp.org/news/claude-code-handbook/

6. https://www.humanlayer.dev/blog/writing-a-good-claude-md
7. https://nervegna.substack.com/p/claude-code-for-designers-a-practical
8. https://theyawns.com/2026/03/06/launching-a-claude-code-project-design-first-then-build/
9. https://platform.claude.com/docs/en/home
10. https://support.claude.com/en/articles/14604397-set-up-your-design-system-in-claude-design
11. https://www-cdn.anthropic.com/58284b19e702b49db9302d5b6f135ad8871e7658.pdf
12. https://academy.claude.com/courses/claude-101/introduction-to-projects
13. https://claude.com/product/claude-code
14. https://readme.com/blog/what-is-claude-md
15. https://www.intodesignsystems.com/claude-code-for-designers
16. https://help.figma.com/hc/en-us/articles/39888612464151-Claude-Code-and-Figma-Set-up-the-MCP-server
17. https://help.figma.com/hc/en-us/articles/32132100833559-Guide-to-the-Figma-MCP-server
18. https://code.claude.com/docs/en/memory
19. https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices
20. https://github.com/figma/mcp-server-guide
21. https://resources.anthropic.com/hubfs/The-Complete-Guide-to-Building-Skill-for-Claude.pdf
22. https://claude.com/blog/skills-explained
23. https://composio.dev/toolkits/figma/framework/claude-code
24. https://github.com/anthropics/skills
25. https://vectorize.io/articles/claude-code-memory
26. https://github.com/luongnv89/claude-howto/blob/main/02-memory/README.md
27. https://code.claude.com/docs/en/skills
28. https://www.figma.com/mcp-catalog/
29. https://www.figma.com/blog/introducing-claude-code-to-figma/
30. https://support.claude.com/en/articles/12123587-import-and-export-your-memory-from-claude
31. https://code.claude.com/docs/en/skills
32. https://help.figma.com/hc/en-us/articles/39888612464151-Claude-Code-and-Figma-Set-up-the-MCP-server
33. https://code.claude.com/docs/en/mcp
34. https://code.claude.com/docs/en/mcp
35. https://help.figma.com/hc/en-us/articles/39888612464151-Claude-Code-and-Figma-Set-up-the-MCP-server

provide comand to claud code to excute on this and also anlyse current semester resporty and specfic to it

Your current Semester repo is already much further along than a generic “add a design system” setup. It has a dedicated token source/export path, extensive style-contract tests, an existing root `CLAUDE.md`, vendored Claude skills, and a large Vite/React application under `app/`. The best next move is **not** to replace that architecture—it is to add a Semester-specific Claude Code audit skill and tighten the existing token-export → Figma-variable → CI loop. [github_mcp_direct]

Run this in Claude Code

From the repository root, launch Claude Code:

```
cd /path/to/semester
claude
```

Then paste this command/prompt exactly into Claude Code:

```
/audit-semester-design-sync app/src app/design-tokens/semester.tokens.json app/src/s

You are auditing the existing Semester repository, not designing a generic new syste

First read:
- CLAUDE.md
- app/README.md
- DESIGN-SYSTEM-GUIDE.md
- docs/DESIGN-TOKENS.md
- docs/design/DESIGN-SYSTEM-PRODUCT-SPEC.md
- app/package.json
- app/src/styles/tokens.css
- app/src/lib/tokenexport.ts
- app/src/lib/tokenexport.test.ts
- app/design-tokens/semester.tokens.json
- app/src/styles/*.test.ts
- .claude/skills/

Then inspect:
- app/src/components/
- app/src/screens/
- app/src/styles/
- app/src/App.tsx
- app/src/main.tsx
- app/src/screens.tsx
```

```
Do not modify files yet.

Create a repository-specific Semester design-system audit with:
1. Existing architecture and source-of-truth map.
2. Token pipeline map: CSS semantic tokens → tokenexport.ts → semester.tokens.json →
3. Current design-system strengths that must be preserved.
4. Drift risks: raw styles, direct hex usage, duplicated component styles, hard-code
5. Existing tests/scripts that already enforce design contracts.
6. Exact missing files needed for Claude Code + Figma MCP workflow only.
7. A minimal phased implementation plan that avoids restructuring the current app.
8. A table of proposed changes with priority, exact path, rationale, and acceptance

Use evidence from actual files. Do not assume pnpm, monorepos, Storybook, Tailwind, c
```

That audit prompt deliberately tells Claude not to invent a `packages/ui`, pnpm workspace, Tailwind setup, or a replacement token schema—the patterns in earlier generic templates would be wrong for this particular repository.

One-command bootstrap prompt

Once the audit completes and you agree with its plan, paste the following command to have Claude Code implement only the smallest compatible toolkit:

```
Implement the approved Semester Claude Code design-system toolkit in this repository

Constraints:
- Preserve the current Vite + React architecture under app/.
- Preserve app/src/styles/tokens.css as the semantic token authority.
- Preserve app/src/lib/tokenexport.ts as the token export authority.
- Preserve app/design-tokens/semester.tokens.json as generated output; never hand-edi
- Preserve and extend existing tokenexport, style, accessibility, responsive, and cor
- Do not add Tailwind, Storybook, a monorepo, or a new component library package.
- Do not move existing components or rewrite App.tsx unless absolutely necessary.
- Do not add dependencies unless an existing repository dependency cannot support the
- Do not commit, push, open a PR, or change deployment configuration.

Create only:
1. .claude/skills/audit-semester-design-sync/SKILL.md
2. .claude/skills/build-semester-ui/SKILL.md
3. .mcp.json using the Figma remote MCP server, with no credentials committed
4. docs/FIGMA-MAPPING.md
5. app/scripts/design-system-audit.mjs or .ts, compatible with the app's existing too
6. app/scripts/design-system-report.mjs or .ts
7. Minimal package.json scripts in app/package.json:
   - design-system:audit
   - design-system:report
   - design-system:check
8. Focused tests that prove:
   - semester.tokens.json is generated and synchronized with code
   - raw values are not introduced outside approved CSS/token files
   - Figma mapping entries resolve to exported token paths
   - audit/report scripts fail on real violations and pass on valid fixtures

Before editing, show the file-by-file plan.
```

```
After editing, run the repository's actual relevant validation commands.
Report changed files, exact checks run, results, known limitations, and any Figma ste
```

Current Semester findings

Architecture already present

The active application is a Vite/TypeScript React app located in `app/`, not a package-per-component design-system monorepo. It has a large app entry point, screen hierarchy, component directory, styling directory, API/server code, and build configuration. [github_mcp_direct]

Current asset	What it means for your toolkit
`app/src/styles/tokens.css`	This should remain the **semantic-token authority**
`app/src/lib/tokenexport.ts`	This should remain the **token-export authority**
`app/design-tokens/semester.tokens.json`	This is the machine-readable/exported artifact; do not hand-edit it
`app/src/lib/tokenexport.test.ts`	Already validates that token export does not drift from code
`app/src/styles/*.test.ts`	You already have a substantial contract-test surface for CSS design rules
`app/src/components/` and `app/src/screens/`	These are the correct places to audit real UI composition and component duplication
`.claude/skills/`	Claude Code skill infrastructure already exists, so add one focused skill rather than bootstrapping from zero
`CLAUDE.md`	Existing durable project instructions should be amended, not replaced blindly
`DESIGN-SYSTEM-GUIDE.md` and `docs/DESIGN-TOKENS.md`	Existing written design-system documentation that new Claude rules should reference

The repository's token documentation explicitly identifies `app/design-tokens/semester.tokens.json` as a generated, machine-readable export; `tokenexport.test.ts` verifies that it remains synchronized with `tokens.css`. That is a stronger foundation than a fresh generic JSON schema because it already supports downstream Figma/native/parity use cases. [github_mcp_direct]

Important constraints

Your project has a **large existing CSS system**: `app.css` is approximately 322 KB, `features.css` about 29 KB, `unity.css` about 34 KB, and `tokens.css` roughly 13 KB. It also has targeted tests for breakpoints, density, motion, fields, glass effects, gutters, hex handling, stacking, taps, typography scale, and token behavior. Treat this as a mature contract system, not a greenfield UI package. [github_mcp_direct]

So Claude Code should:

- Reuse the current CSS token vocabulary before introducing any new token name.
- Extend current source-based token export rather than create duplicate TypeScript token files.
- Use the existing style tests as enforcement hooks instead of adding broad, untested rules in a new lint stack.
- Keep the generated token JSON as the Figma/native interchange artifact.
- Add a Figma mapping manifest and a diff report around the existing export, rather than trying to make Figma the production source of truth.

Existing Figma readiness

The repository contains Figma-aware documentation and token-export comments, but the code search did **not** reveal a configured Figma MCP server or a maintained Figma-to-code component/variable mapping file. The export code and tests already describe `semester.tokens.json` as the artifact intended for Figma variables, native clients, and parity checking. [github_mcp_direct]

That means your gap is integration/governance—not token invention.

Add this audit skill

Create this folder and file:

```
mkdir -p .claude/skills/audit-semester-design-sync
```

```
.claude/skills/audit-semester-design-sync/SKILL.md
```

Paste this exact content:

```
---
name: audit-semester-design-sync
description: Audit the Semester React application against its existing CSS token sys
argument-hint: "[optional-figma-url-or-node] [optional-app-path]"
---

# Audit Semester Design Sync

## Semester-specific source of truth

Semester is an existing Vite and React application. Do not impose a generic monorepo
Tailwind, Storybook, or package-based design-system architecture unless it already ex

The source-of-truth order is:

1. `app/src/styles/tokens.css`
   - Semantic CSS token authority used by application UI.

2. `app/src/lib/tokenexport.ts`
```

```
   - Exports the application's semantic token system for downstream consumers.

3. `app/design-tokens/semester.tokens.json`
   - Generated machine-readable token artifact.
   - Never edit manually.
   - Regenerate through the repository's actual token-export command.

4. Existing CSS/component implementation
   - `app/src/styles/`
   - `app/src/components/`
   - `app/src/screens/`

5. Existing contract tests
   - `app/src/styles/*.test.ts`
   - `app/src/lib/tokenexport.test.ts`
   - Related architecture, responsive, accessibility, and screen tests.

6. Figma
   - Figma is evidence of approved visual intent.
   - Code owns production behavior, accessibility, responsive implementation, and tol
   - Figma variables/components must map to existing exported token paths and React/C

## Required reading

Before auditing, read:

- `CLAUDE.md`
- `app/README.md`
- `DESIGN-SYSTEM-GUIDE.md`
- `docs/DESIGN-TOKENS.md`
- `docs/design/DESIGN-SYSTEM-PRODUCT-SPEC.md`
- `app/package.json`
- `app/src/styles/tokens.css`
- `app/src/lib/tokenexport.ts`
- `app/src/lib/tokenexport.test.ts`
- `app/design-tokens/semester.tokens.json`
- `app/src/styles/*.test.ts`
- `docs/FIGMA-MAPPING.md`, if it exists

For a UI-specific audit, inspect the target implementation plus nearest related scree
components, and styles.

## Figma workflow

If a Figma URL, selection, node, component, or variable is supplied:

1. Use configured Figma MCP tools to inspect:
   - Node hierarchy.
   - Figma variables and collections.
   - Component instances and properties.
   - Auto-layout, constraints, and visible states.
   - Text/content hierarchy.

2. Compare Figma variable names to:
   - `app/design-tokens/semester.tokens.json`
   - `docs/FIGMA-MAPPING.md`
```

```
   - Semantic names in `app/src/styles/tokens.css`

3. Compare Figma components to current React/CSS components.
4. Do not create raw CSS to obtain pixel-perfect parity.
5. Do not treat an incomplete Figma frame as permission to omit:
   - Keyboard support.
   - Focus visibility.
   - Empty/loading/error states.
   - Responsive behavior.
   - Accessible labels and semantic HTML.

## Audit checks

### Token integrity

Identify:

- Direct hex, rgb, hsl, oklch, arbitrary shadows, arbitrary spacing, arbitrary radii
  arbitrary z-indexes, arbitrary durations, or arbitrary easing values.
- Whether each raw value is permitted in a foundational source file or is an app-leve
- CSS variables used but absent from `tokens.css`.
- CSS tokens present but not represented in the generated export.
- Generated token paths that cannot be traced back to semantic CSS tokens.
- Figma variables that lack an exported token mapping.
- Exported semantic tokens that lack an approved Figma mapping, if parity is required

### Component and pattern integrity

Identify:

- Feature-local components duplicating existing component or CSS patterns.
- Hard-coded styling that bypasses existing classes or semantic variables.
- Missing or inconsistent states.
- Misused controls: buttons vs links, clickable divs, unlabeled icon controls.
- Inconsistent density, hierarchy, type scale, surface treatment, borders, and page g
- Component APIs or CSS selectors that leak implementation detail.

### Responsive integrity

Check:

- Current breakpoint/test contracts.
- Narrow, intermediate, and wide behavior.
- Overflow, scroll, truncation, long copy, localization, and touch targets.
- Fixed/sticky element and stacking behavior.
- Data/table fallback behavior on narrow layouts.

### Accessibility integrity

Check:

- Semantic landmarks and headings.
- Label, description, error, and input associations.
- Keyboard interaction and visible focus.
- Dialog/menu/popover focus behavior.
- Color contrast and non-color status communication.
```

```
- Reduced-motion behavior.
- Target sizing.

## Severity scale

- Blocker: breaks critical usability, accessibility, core system integrity, or releas
- Major: repeatable drift, broken component/pattern contract, significant responsive,
- Minor: localized inconsistency, documentation gap, or low-risk visual mismatch.

## No-edit audit mode

Do not modify source code during an audit unless the user explicitly asks after revie
the report. You may run read-only inspection commands and existing test/audit commanc

## Required report format

# Semester design-system audit

## Executive assessment

State:
- Overall alignment: strong, moderate, weak, or critical drift.
- Highest-risk finding.
- Whether the main concern is tokens, components, responsive behavior, accessibility
  Figma parity, or test coverage.

## Repository model

| Concern | Current Semester authority | Evidence |
|---|---|---|
| Semantic tokens | `app/src/styles/tokens.css` | Specific section/file |
| Token export | `app/src/lib/tokenexport.ts` | Specific function/file |
| Machine-readable artifact | `app/design-tokens/semester.tokens.json` | Specific fi
| UI implementation | Components/screens/styles | Specific paths |
| Enforcement | Existing tests/scripts | Exact script or test paths |
| Figma parity | Mapping file or missing | Evidence |

## Findings

| Severity | Location | Category | Evidence | Existing Semester solution | Smallest s
|---|---|---|---|---|---|
| Blocker/Major/Minor | Exact path, selector, token, or Figma node | Token/Component/

## Token parity

| Figma variable or code token | Semester exported path | CSS semantic source | Statu
|---|---|---|---|---|
| Value | Value | Value | Match/Partial/Missing/Obsolete | Specific resolution |

## Component parity

| Figma or screen component | Current React/CSS implementation | State/variant parity
|---|---|---|---|---|
| Value | Value | Match/Partial/Missing | Match/Partial/Missing/Obsolete | Specific r

## Existing enforcement
```

```
| Contract | Existing test/script | Coverage | Gap |
|---|---|---|---|
| Token export | Exact test/script | What it proves | What remains |
| Responsive | Exact test/script | What it proves | What remains |
| Accessibility | Exact test/script | What it proves | What remains |
| Raw values | Exact test/script | What it proves | What remains |

## Minimal implementation plan

| Priority | Exact path | Change | Why | Acceptance test |
|---|---|---|---|---|
| P0/P1/P2 | Exact repo path | Smallest concrete change | System value | Exact commar

## Rules for recommendations

- Preserve Semester's existing token and export architecture.
- Recommend extensions, not parallel sources of truth.
- Use current tests before introducing tooling.
- Do not recommend Tailwind, Storybook, a monorepo, or package migrations unless
  directly supported by evidence and explicitly requested.
- Do not create a component for a one-off visual difference.
- Never recommend direct raw values to match Figma pixels.
- Prefer semantic CSS variables, existing classes, documented CSS patterns, and curre
```

Figma MCP setup

For Semester, configure the **remote** Figma MCP endpoint at the repository level, but do not commit credentials.

Run from the repository root:

```
claude mcp add --scope project --transport http figma https://mcp.figma.com/mcp
```

Then launch Claude Code:

```
claude
```

In Claude Code:

```
/mcp
```

```
Authorize Figma through the browser prompt, then verify Figma is connected. Claude Co
```

If you prefer a committed project declaration, add a root `.mcp.json`:

```
{
  "mcpServers": {
    "figma": {
```

```
      "type": "http",
      "url": "https://mcp.figma.com/mcp"
    }
  }
}
```

The `type: "http"` field matters: Claude Code needs it to interpret the URL as a remote HTTP MCP server instead of an attempted local stdio server.[1]

Figma recommends its official Claude Code plugin as the lowest-friction approach, because the plugin includes the MCP configuration and Figma-related workflow skills:

```
claude plugin install figma@claude-plugins-official
```

Use **one** setup route initially:

- Prefer the official plugin if you want Figma's maintained defaults and bundled workflows.
- Prefer the checked-in `.mcp.json` if you need a transparent, reviewable repository configuration.
- Avoid configuring duplicate servers under multiple names until you have a real reason; duplicated MCP tools make Claude's tool selection less reliable. Figma MCP gives Claude structured access to variables, components, layout information, and other design context used for implementation and audit work.[2]

What to add—not replace

Add now	Why
`.claude/skills/audit-semester-design-sync/SKILL.md`	Gives Claude a repeatable repo-specific design review workflow
`.claude/skills/build-semester-ui/SKILL.md`	Ensures future UI work follows existing tokens, CSS, components, and test contracts
`docs/FIGMA-MAPPING.md`	Makes token/component parity explicit and reviewable
`.mcp.json` or the Figma plugin	Gives Claude structured Figma context
`app/scripts/design-system-audit.*`	Detects prohibited raw styling and missing token paths
`app/scripts/design-system-report.*`	Produces a CI-readable design-system drift summary
Package scripts in `app/package.json`	Makes the audit reproducible locally and in CI
Do not add yet	Why
---	---
A second token source in `packages/tokens/`	Semester already has `tokens.css` + `tokenexport.ts` + generated JSON
A generic `@semester/ui` package	No evidence that your current app uses package-based UI distribution

Add now	Why
Tailwind/token migration	Would create a parallel styling system and increase drift risk
Storybook as a prerequisite	Useful later, but first use your established CSS contracts and app-screen tests
A new component framework	Existing components/screens/styles need an evidence-based audit first
Automatic Figma writes	Figma reads are appropriate for audit; design-file mutations need explicit review

The repository is public and currently identified as `harrisonjrubin7-cmyk/semester`, with `main` as the default branch. It contains roughly 53 open issues and a substantial TypeScript codebase, so start with a **read-only audit and minimal additive configuration**, rather than a sweeping UI-platform rewrite. [github_mcp_direct]

⁂

1. https://code.claude.com/docs/en/mcp
2. https://help.figma.com/hc/en-us/articles/39888612464151-Claude-Code-and-Figma-Set-up-the-MCP-server